Software Requirements Specification

for

HDB Resale Flat Search and Analysis System

Version 1.4

Prepared by Team ABCDE

Roen
Aleesha
Manwen
Jing Yi
Junyao

Nanyang Technological University

2/11/2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Revision History ..... **iv**

1. Introduction **1**

1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 2
1.5 References 2

2. Overall Description **2**

2.1 Product Perspective 2
2.2 Product Functions 3
2.3 User Classes and Characteristics 3
2.4 Operating Environment 4
2.5 Design and Implementation Constraints 4
2.6 User Documentation 4
2.7 Assumptions and Dependencies 5
2.8 System Architecture 5
2.8.1 Architectural style 5
2.8.2 Deployment 7
2.8.3 How the architecture meets the non-functional requirements 7
2.8.4 The batch scheduler and singleton scope 8
2.9 Design Model 8
2.9.1 Package structure 8
2.9.2 Design class diagram 10
2.9.3 Start-up class 14
2.9.4 Container classes 14
2.9.5 Control class responsibilities 15
2.9.6 Refined dialog map 15
2.10 Design Patterns 17
2.11 Key Design Issues 17
2.11.1 Identifying and storing persistent data 17
2.11.2 Providing access control 18
2.11.3 Control flow and boundary conditions 19
2.12 Traceability 19
2.13 Implementation Status (Lab 4) 19

3. External Interface Requirements **20**

3.1 User Interfaces 20
3.2 Hardware Interfaces 23
3.3 Software Interfaces 24
3.4 Communications Interfaces 25

4. System Features **25**

4.1 Search Flats by Dual-Anchor Commute 25
4.1.1 Description and Priority 25
4.1.2 Stimulus/Response Sequences 25
4.1.3 Functional Requirements 26
4.2 Adjust livability weights 27
4.2.1 Description and Priority 27
4.2.2 Stimulus/Response Sequences 27

4.2.3 Functional Requirements....27
4.3 View flat details, valuation and CPF calculator....28
4.3.1 Description and Priority....28
4.3.2 Stimulus/Response Sequences....28
4.3.3 Functional Requirements....29
4.4 Manage bookmarks and compare flats....29
4.4.1 Description and Priority....29
4.4.2 Stimulus/Response Sequences....29
4.4.3 Functional Requirements....30
4.5 Authenticate User....30
4.5.1 Description and Priority....30
4.5.2 Stimulus/Response Sequences....30
4.5.3 Functional Requirements....31
4.6 Update Valuation Baseline (Batch Refresh)....31
4.6.1 Description and Priority....31
4.6.2 Stimulus/Response Sequences....31
4.6.3 Functional Requirements....32
4.7 Manage Profile and Sign Out....32
4.7.1 Description and Priority....32
4.7.2 Stimulus/Response Sequences....32
4.7.3 Functional Requirements....32
5. Other Nonfunctional Requirements....33
5.1 Performance Requirements....33
5.2 Safety Requirements....33
5.3 Security Requirements....33
5.4 Software Quality Attributes....34
5.5 Business Rules....34
6. Other Requirements....35
Appendix A: Data Dictionary....36
Appendix B: Analysis Models....39
Appendix C: To Be Determined List....46

Revision History

Name	Date	Reason For Changes	Version
Roen, Manwen, Jing Yi, Aleesha, Junyao	6/9/2026	Initial Edit of the Document	1.0
Roen, Manwen, Jing Yi, Aleesha, Junyao	16/9/2026	Adding Diagrams and Updating Section 4 (Lab 2 deliverables)	1.1
Roen, Manwen, Jing Yi, Aleesha, Junyao	1/10/2026	Lab 3 deliverables. Completed Sections 2.6, 2.7, 3.3, 3.4 and 6. Aligned Sections 2.4, 2.5 and 3.3 with the chosen technology stack. Added FR-31 to FR-35 and Section 4.7. Updated the use case diagram (B.4). Filled in the TBD list (Appendix C). Corrected Appendix A and Class Diagram B.1 multiplicities, made the Livability Index a calculated value, and defined the FR-6 weight redistribution rule.	1.2
Roen, Manwen, Jing Yi, Aleesha, Junyao	27/10/2026	Lab 3 feedback. Moved the system architecture and design model from the separate Design Model report into Section 2 as Sections 2.8 to 2.13. Updated the references in Sections 1.1, 1.3 and 3.1 and Appendix B.3.	1.3
Roen, Manwen, Jing Yi, Aleesha, Junyao	27/10/2026	Lab 4. Updated Sections 2.8.3, 2.9.4, 2.11.2 and 2.13 to match the implemented application, replacing the Lab 3 skeleton status. Named the CPF Housing Grant in FR-11 alongside the EHG and PHG. Final update on 2/11/2026: recorded the resolved loan slider ranges and the data sources in 2.13, corrected the black-box test count, named the CPF Housing Grant in 1.4, 2.2, 5.5 and Appendix A, reworded NFR-1, and recorded the outcome of every TBD in Appendix C.	1.4

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) documents the requirements for the HDB Resale Flat Search and Analysis System. The system is designed to assist first-time homebuyers, young families, and property buyers in searching and evaluating HDB resale flats by providing dual-workplace commute calculations, a customizable Livability Index, fair-value pricing labels, and CPF housing grant eligibility estimates. This document covers the complete system, including both frontend and backend components. The system architecture and design model produced in Lab 3 are in Sections 2.8 to 2.13.

1.2 Document Conventions

Throughout this document, the following conventions apply:

1. All functional requirements are uniquely identified with the prefix "FR-" followed by a sequential number. FR-31 to FR-35 were added in version 1.2.
2. All non-functional requirements are uniquely identified with the prefix "NFR-" followed by a sequential number.
3. Requirements marked with "TBD" indicate information that is yet to be determined. Every TBD is tracked in Appendix C.
4. Priority levels for system features are indicated as High, Medium, or Low.
5. User interface mockups referenced in this document are low-fidelity wireframes used for requirements elicitation and are not final visual designs.
6. Class names in the design model use UpperCamelCase and UML stereotypes (<<boundary>>, <<control>>, <<entity>>) follow the Boundary-Control-Entity pattern.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following audiences:

1. Developers: Should read Sections 1-4 to understand functional requirements, Section 5 for performance and quality attributes, and Appendix B for analysis models. The system architecture and design model are in Sections 2.8 to 2.13.
2. Project Managers: Should focus on Sections 1, 2, and 5 to understand project scope, constraints, and quality objectives, and Appendix C for open issues.
3. Testers: Should read Sections 3 and 4 in detail to understand interface requirements and functional specifications for test case creation.
4. Users/Stakeholders: Should read Sections 1, 2.2, and 4 for an overview of product functions and features.

Recommended reading sequence: Begin with Sections 1 and 2 for context, then proceed to Section 4 for feature details, followed by Section 3 for interface requirements.

1.4 Product Scope

The HDB Resale Flat Search and Analysis System is a web-based application that helps users search, evaluate, and compare HDB resale flats. The system provides the following key capabilities:

- Dual-workplace commute time calculation using OneMap and LTA DataMall APIs
- Customizable Livability Index based on four factors: remaining lease years, crime rate, residential density, and air quality
- Fair-value pricing labels (Under-Valued, Fair-Valued, Over-Valued) relative to estate averages
- CPF housing grant eligibility estimation (CPF Housing Grant, EHG and PHG) and monthly cash outlay calculation
- Bookmarking and side-by-side comparison of flats

The system aligns with the business goal of helping Singaporean homebuyers make informed property decisions by balancing budget, commute convenience, and long-term livability.

1.5 References

The following documents and resources are referenced in this SRS:

1. SC2006 Lab 1, Lab 2 and Lab 3 Manuals, Nanyang Technological University, 2026
2. Use Cases for HDB Resale Flat Search and Analysis System, Version 1.2, Team ABCDE, 2026
3. OneMap API Documentation, Singapore Land Authority, https://www.onemap.gov.sg/docs/
4. LTA DataMall API Documentation, Land Transport Authority, https://datamall.lta.gov.sg/
5. data.gov.sg HDB Resale Flat Prices Dataset, https://data.gov.sg/dataset/resale-flat-prices
6. SPF Annual Crime Brief, Singapore Police Force
7. URA Master Plan, Urban Redevelopment Authority
8. NEA Pollutant Standards Index API, National Environment Agency
9. C. Fox, Introduction to Software Engineering Design: Processes, Principles and Patterns with UML2
10. B. Bruegge and A. Dutoit, Object-Oriented Software Engineering: Using UML, Patterns and Java

2. Overall Description

2.1 Product Perspective

The HDB Resale Flat Search and Analysis System is a new, self-contained web application developed as part of the SC2006 course. The system is designed to operate independently while integrating with external data sources and APIs. It serves as a specialised decision-support tool for homebuyers, complementing existing property portals by providing unique features such as dual-workplace commute calculations, customizable livability scoring, and integrated CPF grant estimation.

The system interacts with the following external components:

1. OneMap API: for geocoding, public transport routing and map tiles
2. LTA DataMall API: for supplementary transport information such as train service alerts
3. data.gov.sg: for HDB resale transaction data and NEA PSI readings

2.2 Product Functions

The major functions of the system include:

1. Dual-Anchor Commute Search: Users can enter one required workplace and one optional workplace as commute anchors. The system retrieves travel-time estimates, computes a commute score for each flat, and ranks flats by an overall match score that combines commute and livability.
2. Livability Index Customisation: Users can adjust relative weights (0-100%) for four livability factors: remaining lease years, crime rate, residential density, and air quality. The system automatically adjusts weights to maintain a total of 100% and re-ranks results accordingly.
3. Fair-Value Pricing: The system compares each flat's transaction price against the rolling 6-month estate average and labels it as Under-Valued, Fair-Valued, or Over-Valued, with an estimated fair-value price range.
4. CPF Grant and Affordability Calculator: Users can input household income, marital status, and parental address to check CPF Housing Grant, EHG and PHG eligibility, and estimate monthly cash outlay.
5. Bookmarking and Comparison: Users can bookmark flats for later retrieval and compare two or more bookmarked flats side-by-side.
6. Search and Filtering: Users can filter results by flat type, town/planning area, and price range. Results are displayed as both a ranked list and a map view.
7. Account Management: Users can register, sign in, view and manage their saved profile, and sign out.

2.3 User Classes and Characteristics

User Class	Characteristics	Frequency of Use	Key Functions
First-Time Homebuyers	Individuals or couples purchasing their first HDB flat. May have limited knowledge of property evaluation and need guidance on grants and affordability.	High during property search phase (multiple sessions per week)	CPF grant calculation, affordability estimation, Livability Index
Young Families	Couples with young children or planning to start a family. Prioritise safety, environment quality, and proximity to schools.	Medium-high during decision-making phase	Livability Index, valuation labels, commute calculation to workplaces
Property Investors	Users purchasing for investment purposes. Focus on valuation and long-term appreciation.	Low-medium	Fair-value pricing, transaction history analysis

The most important user class is First-Time Homebuyers, as they represent the primary target audience for the system.

2.4 Operating Environment

The software will operate in the following environment:

- Platform: Web-based application accessible via standard web browsers
- Supported Browsers: Latest two major releases of Google Chrome, Apple Safari, and Microsoft Edge
- Viewport Support: Desktop (≥1280px) and mobile (≥375px) viewports
- Server Environment: Java 17 runtime running a Spring Boot application, with a PostgreSQL database (PostGIS extension) and a Redis cache, packaged as Docker containers on a single managed platform
- Network: Internet connection required for API calls and data retrieval
- Operating System: Any OS capable of running supported web browsers (client). Linux for the server.

2.5 Design and Implementation Constraints

The following constraints apply to the development of this system:

1. Technology Stack: React with TypeScript (built with Vite), Tailwind CSS and MapLibre GL JS for the frontend, and Spring Boot on Java 17 (Spring MVC, Spring Data JPA, Spring Security) for the backend REST API. The reasons for this choice are given in the team's technology stack report.
2. Database: PostgreSQL relational database with the PostGIS extension for storing user data, bookmarks, resale transactions and the valuation baseline, with Redis used to cache external API responses
3. API Integration: Must integrate with OneMap API, LTA DataMall API, and data.gov.sg APIs
4. Security: All user financial inputs and parental addresses must be encrypted at rest using AES-256 and transmitted over TLS 1.3
5. Compliance: Must comply with PDPA guidelines for handling personal data
6. Data Caching: Must implement caching for API responses to ensure performance and availability
7. Batch Processing: Automated batch job required to query data.gov.sg every 24 hours for new transaction records
8. Version Control: Source code must be kept in the team's GitHub repository, with each lab's deliverables in its own folder (for example, lab3).

2.6 User Documentation

The following user documentation will be delivered with the software:

1. In-app help: a short first-use walkthrough of the Search screen, and tooltips that explain each livability factor, what each valuation label means, and the loan assumptions used by the CPF calculator.

2. FAQ page: a web page inside the application that answers common questions, such as where the data comes from and how often it is updated.
3. Disclaimers: clear notices on the Flat Detail screen that prices, grants and cash outlay are estimates and that final grant amounts are decided by HDB.
4. Developer documentation: a README in the GitHub repository with setup and run instructions, and Javadoc (backend) and TSDoc (frontend) comments for all classes and key public methods.

All user documentation is delivered online in English as part of the web application. No printed manual is provided.

2.7 Assumptions and Dependencies

The following assumptions and dependencies could affect the requirements in this SRS:

1. A-1: The OneMap routing API can return public transport travel times for any Singapore address, and the team can obtain a free developer access token.
2. A-2: LTA DataMall provides supplementary transport data (for example, train service alerts) through a free AccountKey.
3. A-3: data.gov.sg continues to publish the HDB resale flat prices dataset and the NEA PSI readings through its public API.
4. A-4: There is no public API for live resale listings. Flats shown by the system are therefore based on recent resale transactions, and "price" refers to the most recent transaction price for that flat.
5. A-5: Crime statistics (SPF), residential density (URA and SingStat) and school locations (MOE) change slowly. They are loaded from static CSV files at start-up and updated by hand when a new edition is published.
6. A-6: CPF grant rules (EHG and PHG) are taken from the HDB website at the time of release and stored in a configuration file.
7. A-7: All external APIs are subject to their providers’ rate limits and terms of use. The system depends on these limits being high enough for 50 concurrent users when combined with caching.
8. A-8: The project depends on open-source libraries (Spring Boot, React, MapLibre GL JS, Redis and their dependencies) remaining available and maintained.

2.8 System Architecture

2.8.1 Architectural style

The system is a client and server web application. The client is a single page React app that runs in the browser. The server is one Spring Boot application split into four layers. Each layer only calls the layer below it, so REST controllers never touch a repository directly and services never deal with HTTP. This keeps coupling low and lets us change one layer, such as swapping an external data provider, without touching the others.

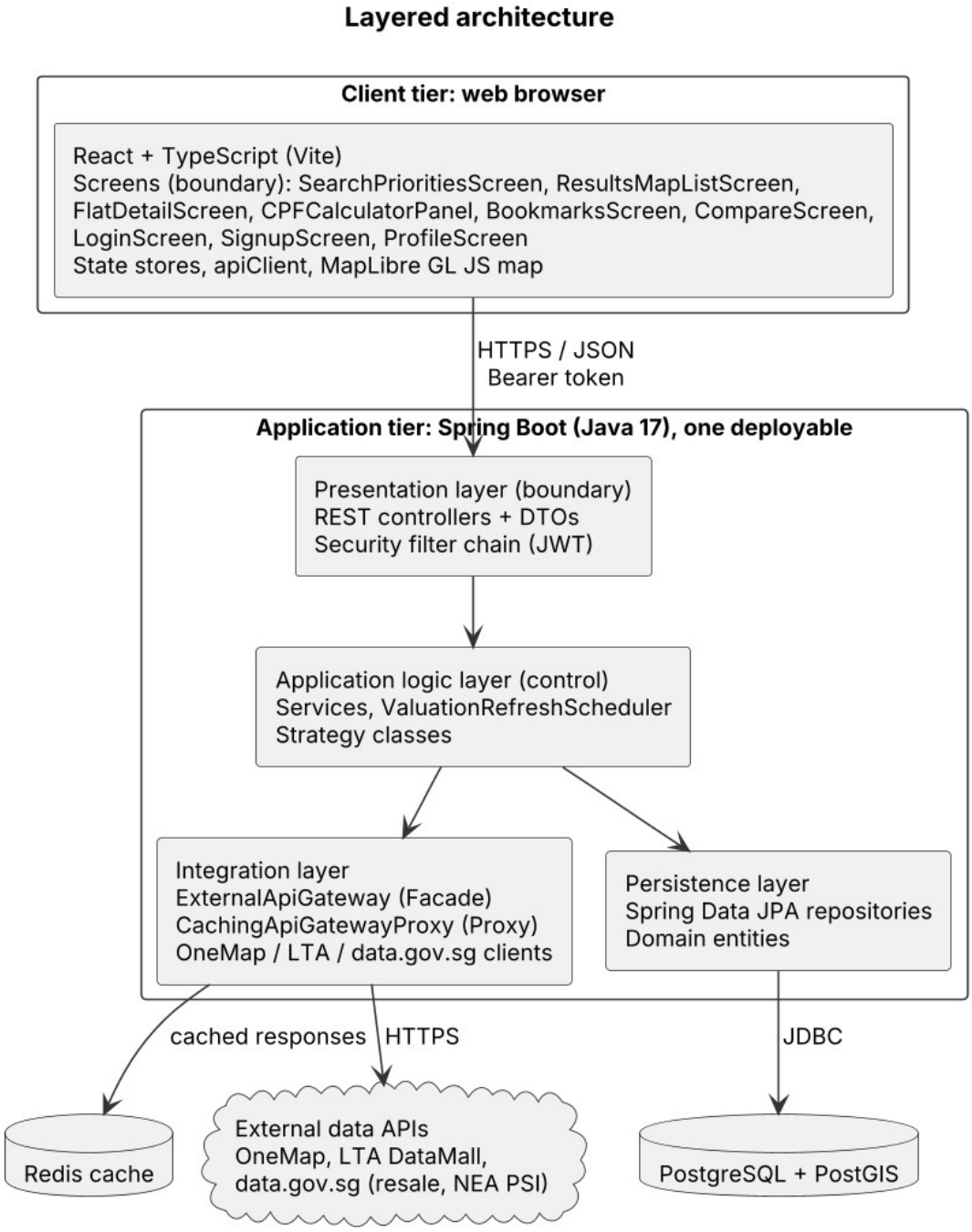


Figure 1: Layered architecture

Layer	Responsibility	Main classes
Client (boundary)	Screens, panels and the map. Holds the current search and weights in state stores and calls the REST API.	React screens, searchStore, authStore, apiClient
Presentation (boundary)	Exposes the REST API, checks request formats, maps exceptions to error replies and applies the security filter chain.	controller package, dto package, GlobalExceptionHandler, SecurityConfig

Layer	Responsibility	Main classes
Application logic (control)	Carries out each use case and holds the business rules.	service package, service.strategy package, ValuationRefreshScheduler
Integration	One entry point to all government APIs, with caching, a 2 second timeout and fallback to the last good value.	ExternalApiGateway, CachingApiGatewayProxy, ApiGatewayImpl, clients
Persistence	Reads and writes entities.	repository package, domain package

2.8.2 Deployment

For production the system runs as three containers started with docker compose. The app container holds one jar that serves both the REST API and the built React files, so there is only one origin and no cross-origin setup in production. PostgreSQL with PostGIS stores the data and Redis caches external API replies. For development the same jar runs on its own with an embedded H2 database and an in-memory cache, so a team member only needs Java and Node installed.

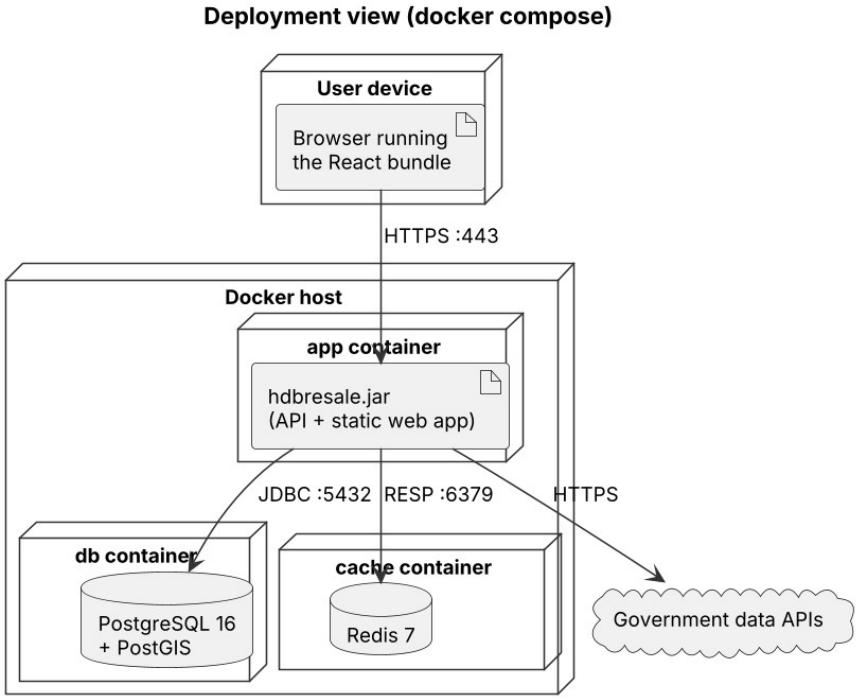


Figure 2: Deployment view

2.8.3 How the architecture meets the non-functional requirements

Requirement	Architectural decision
NFR-1: results for up to 500 flats in 3 seconds	Travel times and geocoding results are cached. A search keeps its scored flats in SearchSessionStore, so filtering and re-weighting re-rank them in memory without calling any external API again.
NFR-3: keep working when an API	CachingApiGatewayProxy waits at most 2 seconds, then returns the last

Requirement	Architectural decision
is slow or down	good value and marks the result as possibly outdated so the screen can show a warning.
NFR-5 and FR-28 to FR-30: daily batch refresh	ValuationRefreshScheduler runs ValuationService.runBatch every day at 03:00 Singapore time. Each run is logged as a ValuationBatchJob, and a failed run keeps the previous baseline.
NFR-6: protect sensitive data	Income and parents' postal code are encrypted with AES-256 before they are stored. All traffic goes over HTTPS.
Maintainability	Layered packages, interfaces for the parts most likely to change (scoring, valuation, external data) and constructor injection so each class can be unit tested on its own.

2.8.4 The batch scheduler and singleton scope

ValuationRefreshScheduler is a Spring bean with the default singleton scope. That means Spring creates exactly one instance of it inside each running application, so one application instance never starts two overlapping refreshes. It does not stop two separate instances of the application from both running the job. Our planned deployment runs a single app container, so this is enough for now. If we ever run more than one instance, the job would need extra coordination, such as a database lock, so that only one instance runs it each day.

2.9 Design Model

2.9.1 Package structure

The backend is one Java package, com.abcde.hdbresale, split into sub-packages that follow the BCE stereotypes and the patterns we used. Figure 3 shows how they depend on each other.

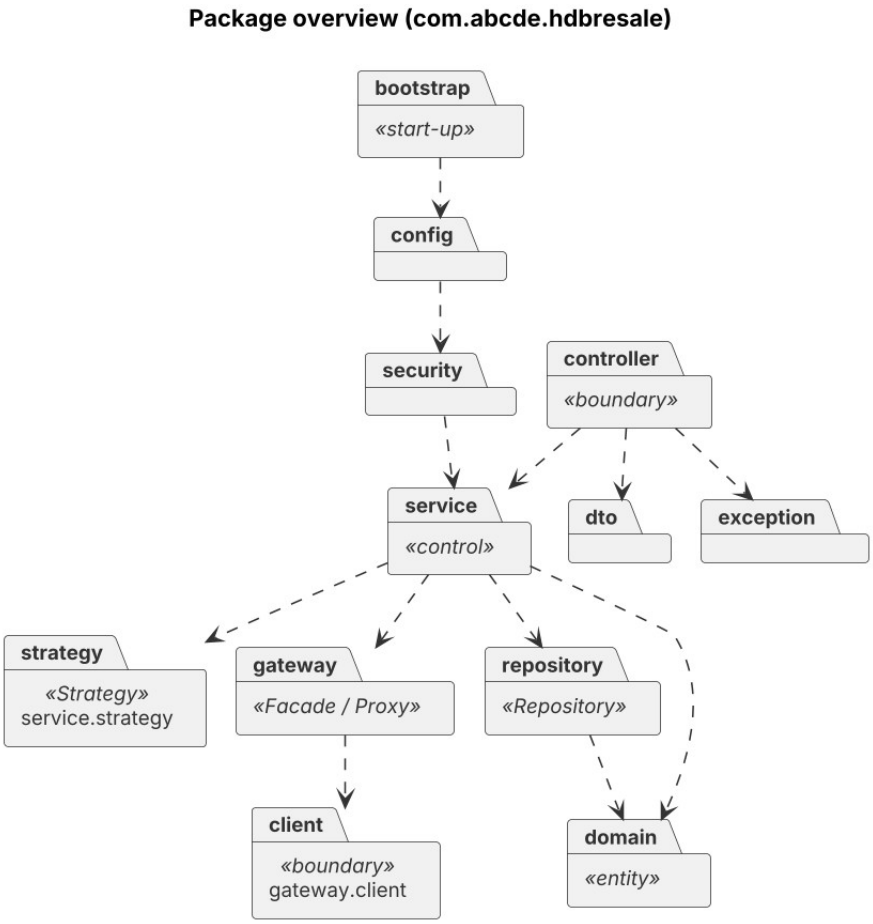


Figure 3: Package overview

Package	**Contents**
bootstrap	Start-up classes: Application and DataSeeder.
controller	REST boundary classes, one per area of the API.
dto	Request and response objects that form the API contract.
service	Control classes, one or more per use case, plus SearchSessionStore.
service.strategy	The two Strategy interfaces and their implementations.
gateway, gateway.client	The Facade and Proxy for external data, and one client per government API.
repository	Spring Data JPA repositories, one per persisted entity.
domain	Entities, value objects and enums.
security	JWT handling, the request filters and AES encryption.
config, exception	Spring configuration, typed settings, grant rules and error handling.

The class diagram is split into five parts so that each one is readable on a page. Classes that appear in more than one part are shown with their full details only once.

2.9.2 Design class diagram

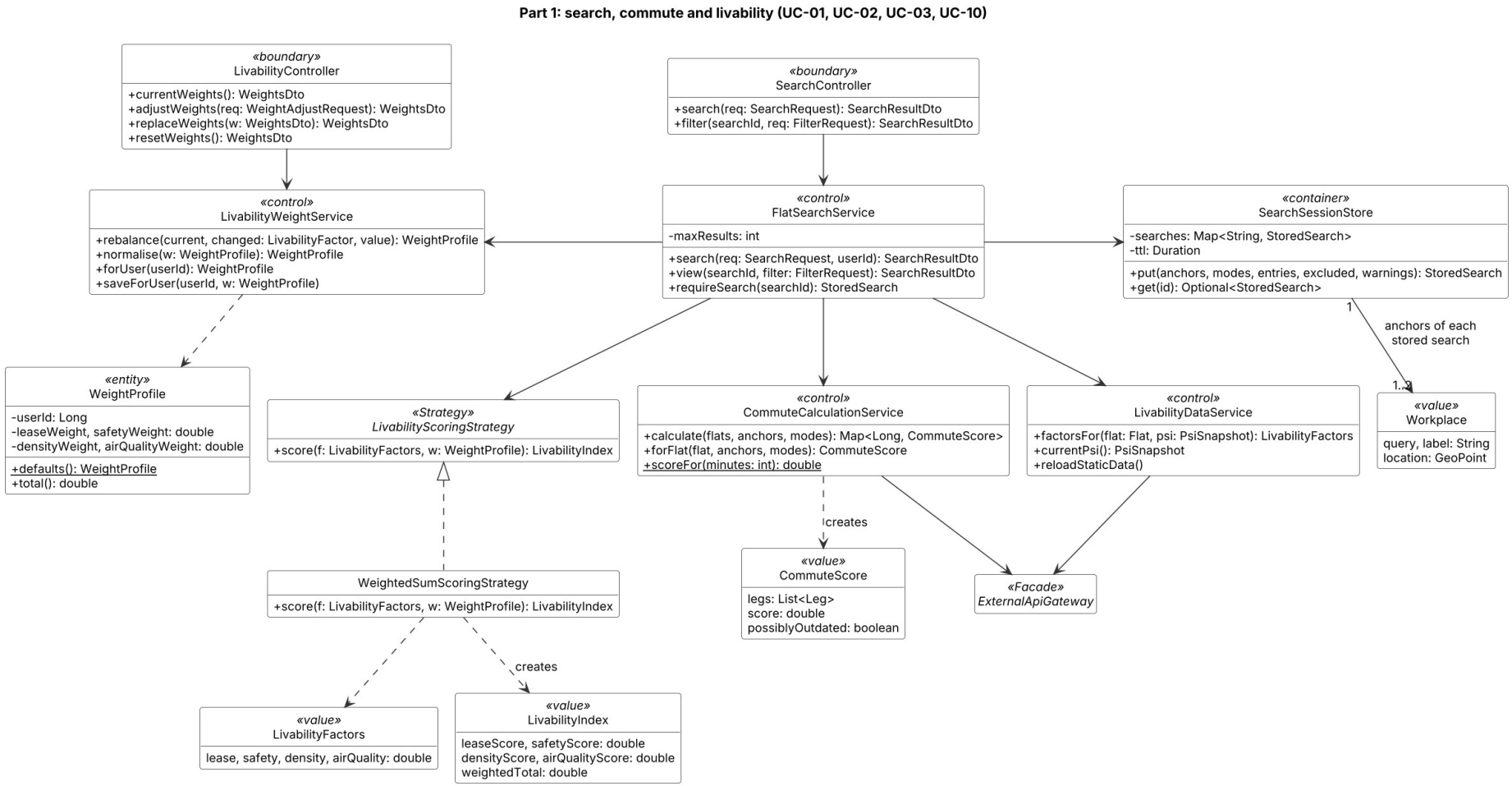


Figure 4: Class diagram part 1: search, commute and livability

A search starts at SearchController, which passes it to FlatSearchService. FlatSearchService geocodes the workplaces, asks CommuteCalculationService for travel times, asks LivabilityDataService for each flat's four factor scores, and ranks the flats. The scored flats are kept in SearchSessionStore so that later filters and weight changes do not repeat the external API calls.

The Livability Index is not a fixed property of a flat. Two users can give the same flat different scores, because the index is always worked out from the flat's four factor scores and the current user's own weights. LivabilityScoringStrategy.score therefore takes both the factors and the WeightProfile, and returns a new LivabilityIndex value each time. Nothing about it is stored.

When the user moves one slider, LivabilityWeightService.rebalance sets that factor and shares the rest of the 100% between the other three in proportion to their current values. If the other three are all 0, they share it equally. Values are rounded to one decimal place and any rounding difference goes to the largest of the other three, so the total is always exactly 100%. This is the rule for FR-6 and UC-03.

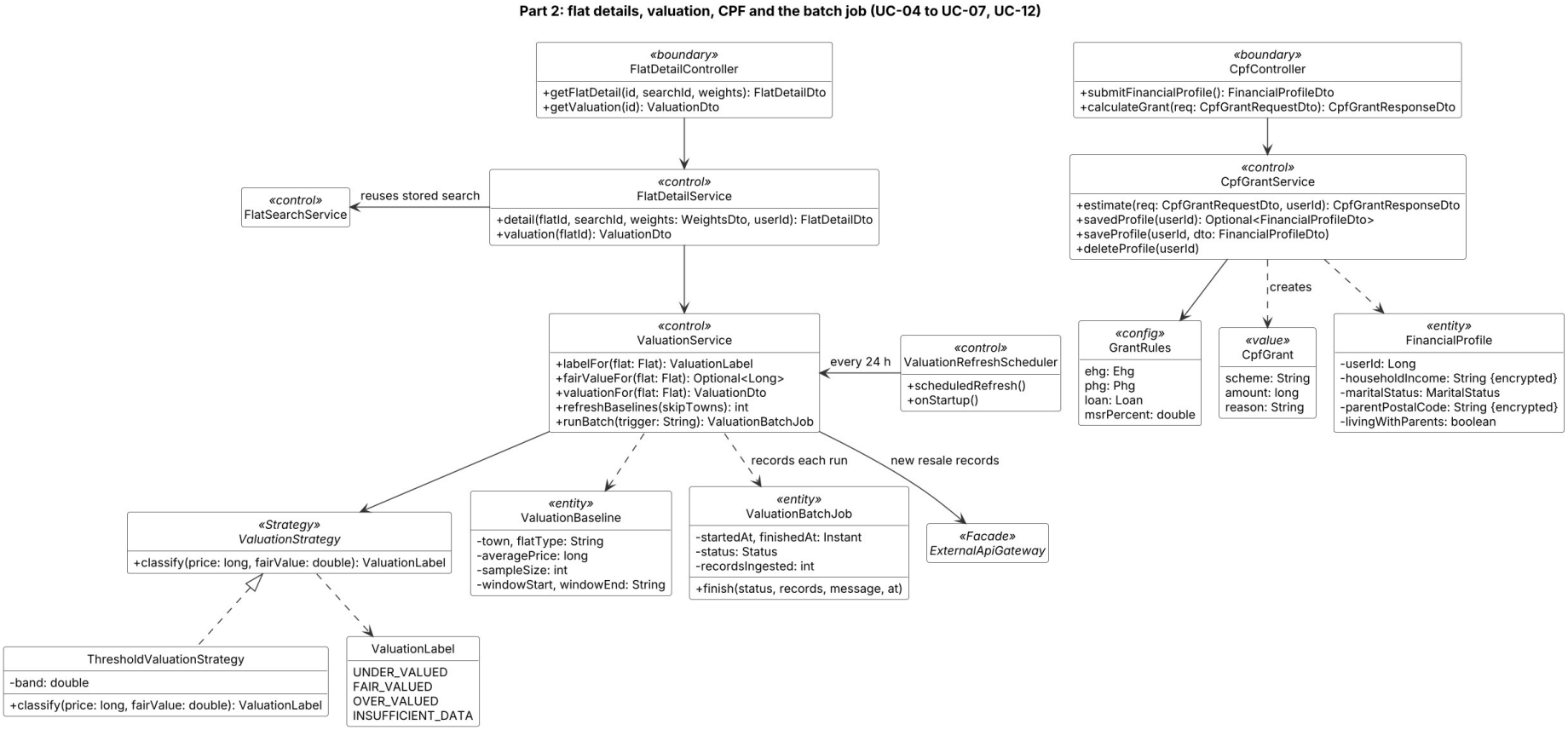


Figure 5: Class diagram part 2: flat details, valuation, CPF and the batch job

ValuationService labels a flat by comparing its latest price with the rolling 6-month average for its town and flat type. The comparison itself sits behind ValuationStrategy, so the 5% band (TBD-1) can change without touching the service. CpfGrantService reads the grant amounts from GrantRules, which is loaded from grant-rules.yml, so new HDB grant figures only need a configuration change.

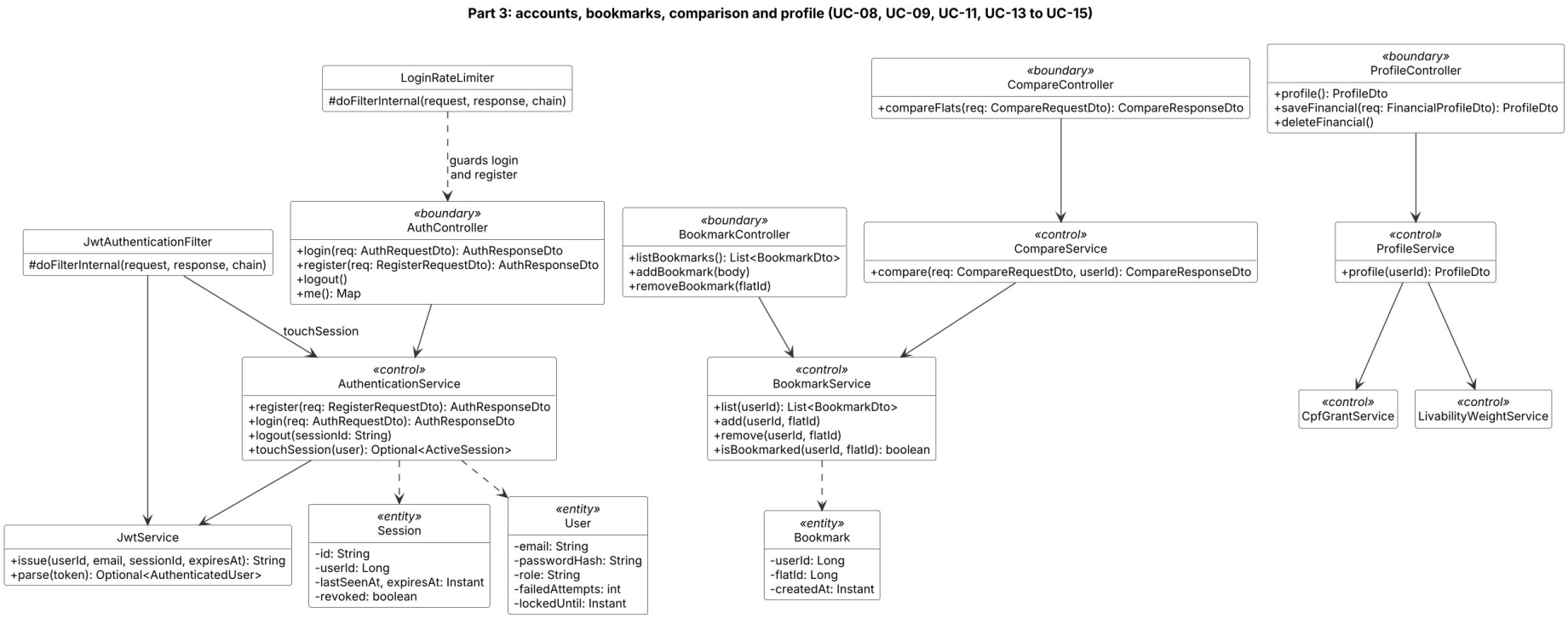


Figure 6: Class diagram part 3: accounts, bookmarks, comparison and profile

AuthenticationService handles register, login and sign out. A JWT is issued at login, and each request passes through JwtAuthenticationFilter, which checks the token and the matching server-side Session. Keeping a Session record means sign out (UC-15) can end a session straight away rather than waiting for the token to expire. CompareService reuses BookmarkService, and ProfileService gathers the account, saved weights and financial profile for the Profile screen.

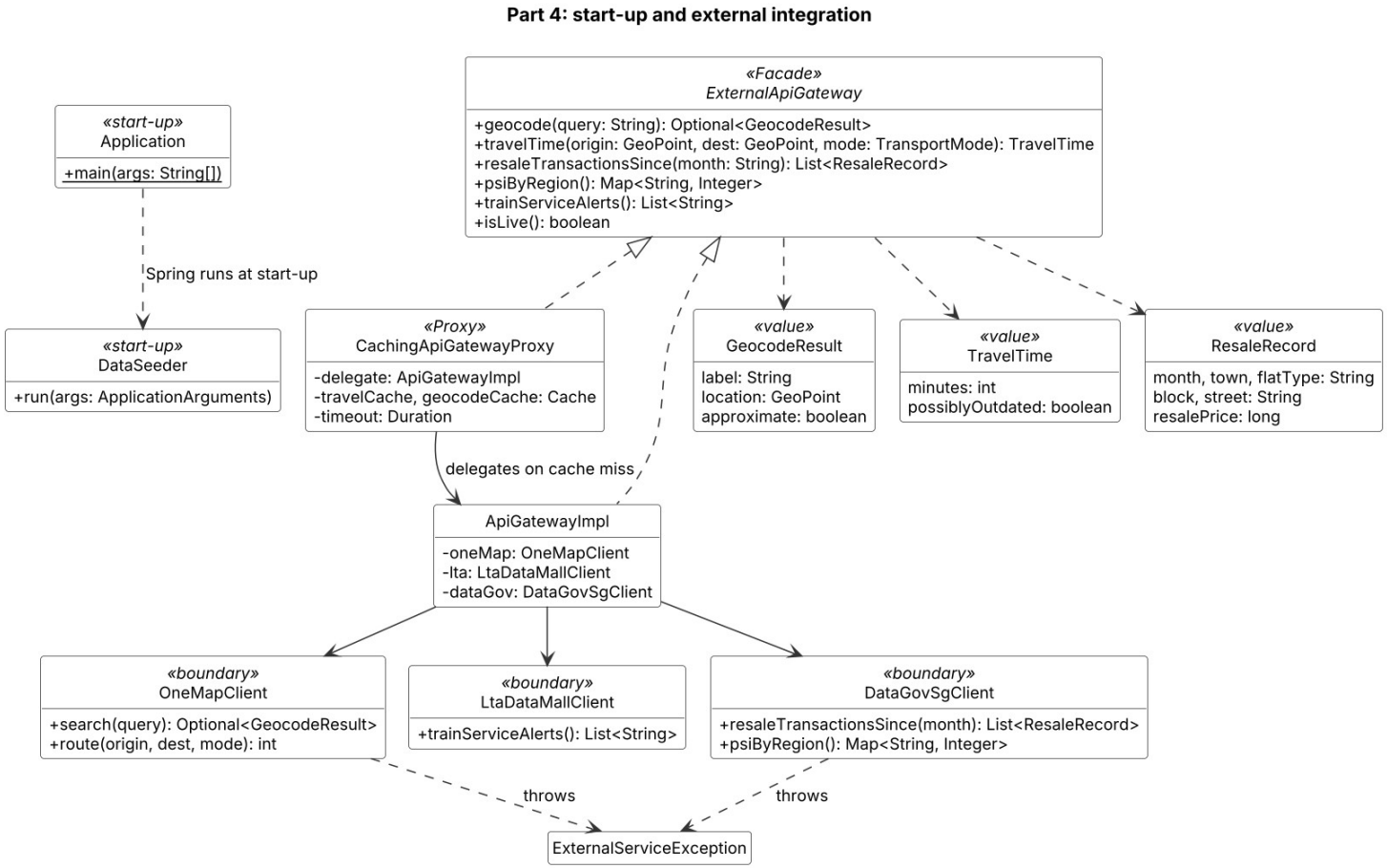


Figure 7: Class diagram part 4: start-up and external integration

Application is the start-up class. It boots the Spring context, which creates and wires every other bean, then DataSeeder loads the static data sets (towns, crime, density, schools) on first start. Services only ever see the ExternalApiGateway interface. The bean they get is CachingApiGatewayProxy, which checks the cache first and only calls ApiGatewayImpl on a miss.

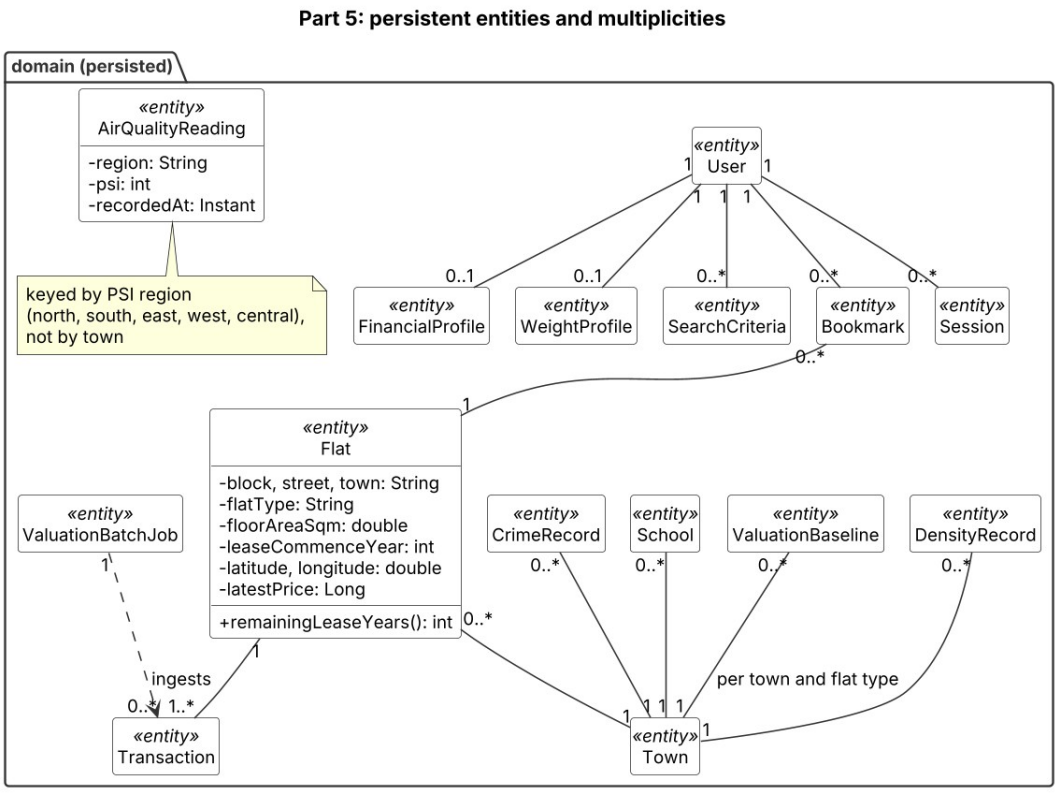


Figure 8: Class diagram part 5: persistent entities and multiplicities

The entities link to each other by id rather than through JPA object references. This keeps each repository query simple and avoids loading a whole graph of objects by accident. The multiplicities above are the logical ones. A user has zero or one saved financial profile and zero or one saved weight profile, because both are only created when the user chooses to save them.

2.9.3 Start-up class

Application holds the main method and starts Spring Boot. Spring then builds every control, boundary and repository object and injects their dependencies through constructors, which is why no class in the design creates its own collaborators with new. DataSeeder runs once the context is ready and loads the CSV data described in SRS assumption A-5. ValuationRefreshScheduler also runs a refresh at start-up if the last successful run is more than 24 hours old.

2.9.4 Container classes

Lab 3 asks for container classes to hold collections of objects. SearchSessionStore is our main container. It holds every recent search in a map keyed by search id, and each stored search holds its list of scored flats, the workplace anchors and any warnings. Entries expire after 60 minutes.

For the other one-to-many associations we follow the mapping rule in the Lab 3 manual and use Java collections directly instead of writing a separate class for each list. Search results travel as a List of RankedFlatDto inside SearchResultDto, bookmarks as a List of BookmarkDto, and a commute score as a List of legs. A dedicated class such as a SearchResultSet or BookmarkList would only wrap a List without adding any behaviour, so we left them out. SearchSessionStore keeps each stored result set in memory and removes it after 60 minutes.

2.9.5 Control class responsibilities

Control class	Use cases	Responsibility
FlatSearchService	UC-01, UC-10	Validates the search, geocodes anchors, scores and ranks flats, applies filters.
CommuteCalculationService	UC-02	Gets travel times for each flat and anchor and turns them into a 0 to 100 commute score.
LivabilityWeightService	UC-03	Rebalances weights to 100% and saves the last-used weights for signed-in users.
LivabilityDataService	UC-01, UC-03	Works out each flat's four factor scores from lease, crime, density and PSI data.
FlatDetailService	UC-04	Assembles the detail view: commute legs, livability breakdown, valuation, schools.
ValuationService	UC-05, UC-12	Labels flats against the rolling average and refreshes the baselines.
CpfGrantService	UC-06, UC-07	Estimates EHG and PHG and the monthly outlay, and saves the financial profile.
BookmarkService	UC-08	Adds, removes and lists bookmarks.
CompareService	UC-09	Builds the side-by-side comparison of two or more flats.
AuthenticationService	UC-11, UC-13, UC-15	Registers accounts, checks credentials, locks accounts, ends sessions.
ProfileService	UC-14	Gathers the account details, saved weights and financial profile.
ValuationRefreshScheduler	UC-12	Triggers the daily batch refresh.

2.9.6 Refined dialog map

The dialog map below refines the initial map in Appendix B.3. Compared with Lab 2 it adds the Sign-Up screen for UC-13, shows the Filter panel and the Adjust Weights panel as panels of the Results screen, shows that the Financial Profile panel can be reached from both the CPF calculator and the Profile screen, and adds the sign-in guard on bookmarking and the sign out transition. Labels follow the form event [guard] / action.

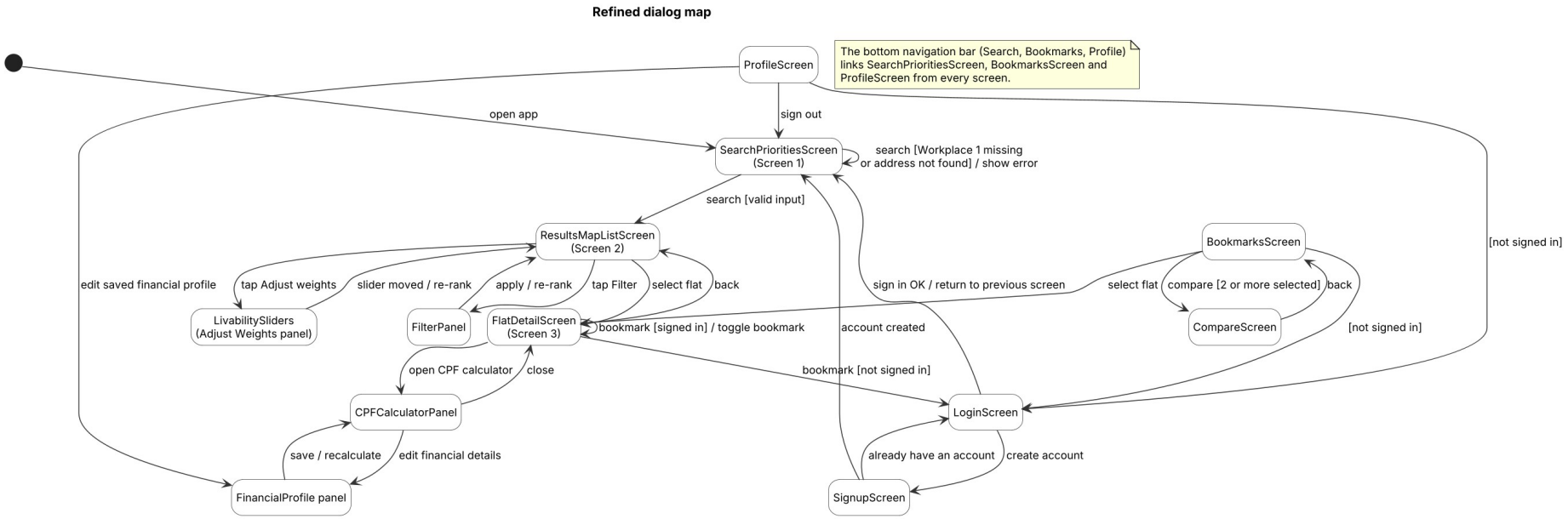


Figure 9: Refined dialog map

2.10 Design Patterns

Pattern	Where it is used	Why
Strategy	LivabilityScoringStrategy with WeightedSumScoringStrategy. ValuationStrategy with ThresholdValuationStrategy.	The scoring formula and the valuation band are both still open questions (TBD-1, TBD-2). A new rule is a new class, and the services do not change.
Facade	ExternalApiGateway	Services get one simple interface to four external data sources instead of depending on each provider's API.
Proxy	CachingApiGatewayProxy	Adds caching, the 2 second timeout and the last-good-value fallback (NFR-1, NFR-3) without changing the real gateway or its callers.
Adapter	OneMapClient, LtaDataMallClient, DataGovSgClient	Each client converts one provider's JSON format into our own types, such as GeocodeResult and ResaleRecord.
Repository	repository package	Hides the database behind interfaces, so services can be tested with mocks and the database can change from H2 to PostgreSQL.
Singleton (Spring scope)	Every service, including ValuationRefreshScheduler	One shared instance per application context. See Section 2.8.4 for what this does and does not guarantee.
Dependency Injection	All constructors	Classes receive their collaborators instead of creating them, which keeps coupling low and makes unit tests easy.
Data Transfer Object	dto package	The API contract can stay stable while entities change, and sensitive fields such as passwordHash never leave the server.

2.11 Key Design Issues

2.11.1 Identifying and storing persistent data

We stored only what has to outlive a request or a server restart. Everything that can be worked out again from stored data and the user's current inputs is calculated when needed. The table below lists what is stored, where it comes from and which repository manages it.

Entity	Source and lifetime	Repository
User, Session	Created at register and login. Sessions end at sign out or after the TBD-5 limits.	UserRepository, SessionRepository
FinancialProfile	Saved only if the user chooses to (UC-07). Income and parents' postal code encrypted.	FinancialProfileRepository
WeightProfile	Last-used weights of a signed-in user (FR-21).	WeightProfileRepository
SearchCriteria	Last search of a signed-in user (FR-21).	SearchCriteriaRepository
Bookmark	Added and removed by the user (UC-08).	BookmarkRepository
Flat, Transaction	Resale transactions from data.gov.sg. A flat is a block and unit type that has appeared in recent transactions. It is not a live listing.	FlatRepository, TransactionRepository
Town, School,	Static CSV data loaded at start-up (SRS A-5).	TownRepository,

Entity	Source and lifetime	Repository
CrimeRecord, DensityRecord		SchoolRepository, CrimeRecordRepository, DensityRecordRepository
AirQualityReading	Latest PSI per region, kept as the fallback when the PSI API is down.	AirQualityReadingRepository
ValuationBaseline	Rolling 6-month average per town and flat type, rebuilt by the batch job.	ValuationBaselineRepository
ValuationBatchJob	One record per batch run, for monitoring and FR-30.	ValuationBatchJobRepository

The following are deliberately not stored. The Livability Index depends on the current user's weights, so it is calculated each time (Section 2.9.2). Commute scores depend on the workplaces in the current search, and the raw travel times are cached in Redis instead. CPF grant estimates depend on the financial details entered at that moment. Search results live in SearchSessionStore for 60 minutes and are then discarded.

The schema is created by a Flyway migration (V1__initial_schema.sql), so every team member and every environment builds the same tables. The production database image includes PostGIS for location queries, while the current schema keeps latitude and longitude as plain numbers.

2.11.2 Providing access control

The system has three kinds of actor. A guest can search and view flats. A signed-in Home Buyer can also save data. The Scheduler is the system itself running the batch job. The access matrix below shows who may do what.

Operation	Guest	Home Buyer	Scheduler
Search, filter, adjust weights (UC-01 to UC-03, UC-10)	Yes	Yes	No
View flat details, valuation (UC-04, UC-05)	Yes	Yes	No
CPF calculator without saving (UC-06)	Yes	Yes	No
Save financial profile (UC-07)	No	Own data only	No
Bookmark and compare (UC-08, UC-09)	No	Own data only	No
Register, sign in (UC-11, UC-13)	Yes	Not needed	No
View and edit profile, sign out (UC-14, UC-15)	No	Own data only	No
Refresh valuation baseline (UC-12)	No	No	Yes

The proposed design uses Spring Security with JWT-based authentication. SecurityConfig marks the bookmark, compare and profile endpoints as requiring a signed-in user, and every other endpoint as open. A signed-in request carries a short-lived token, and JwtAuthenticationFilter checks both the token and the server-side Session before letting it through. Services always take the user id from the verified token, never from the request body, which is how "own data only" is enforced. Passwords are stored as BCrypt hashes. After 5 failed logins an account is locked for 15 minutes (TBD-4), and LoginRateLimiter also limits login attempts per client. Sensitive personal data is encrypted with AES-256 before it is stored (NFR-6).

Issuing and checking tokens, the lockout count, login rate limiting and AES-256-GCM encryption of sensitive fields are implemented in AuthenticationService, LoginRateLimiter and AesEncryptor.

2.11.3 Control flow and boundary conditions

Most of the system is driven by user requests. The one exception is the daily batch job, which Spring's scheduler starts on its own. At start-up the system loads its static data and builds a valuation baseline if none exists. When an external API fails, the gateway falls back to cached data and the screen shows a warning instead of an error. A flat is left out of the results only when there is neither a live nor a cached travel time for it. If the batch job fails, the previous baseline stays in use and the failure is recorded (FR-30).

2.12 Traceability

Every use case in Use Cases v1.2 and every requirement from FR-1 to FR-35 maps to at least one screen, REST endpoint and control class.

Use case	Requirements	Screen	Endpoint	Control classes
UC-01 Search Flats by Dual-Anchor Commute	FR-1 to FR-4, FR-17 to FR-20, FR-22, FR-23, FR-31	SearchPrioritiesScreen, ResultsMapListScreen	POST /api/search	FlatSearchService
UC-02 Calculate Commute Time	FR-3, NFR-3	(inside UC-01)	(internal)	CommuteCalculationService
UC-03 Adjust Livability Weights	FR-5 to FR-8, FR-21	ResultsMapListScreen	POST /api/livability/weights/adjust	LivabilityWeightService
UC-04 View Flat Details	FR-23, NFR-3	FlatDetailScreen	GET /api/flats/{id}	FlatDetailService
UC-05 View Fair-Value Pricing	FR-9, FR-32	FlatDetailScreen	GET /api/flats/{id}/valuation	ValuationService
UC-06 Calculate CPF Grant and Affordability	FR-11 to FR-13	CPFCalculatorPanel	POST /api/cpf/estimate	CpfGrantService
UC-07 Input Financial Profile	FR-10, NFR-6	CPFCalculatorPanel	PUT /api/profile/financial	CpfGrantService
UC-08 Bookmark Flat	FR-14, FR-15, FR-21	FlatDetailScreen, BookmarksScreen	POST, DELETE /api/bookmarks	BookmarkService
UC-09 Compare Bookmarked Flats	FR-16	CompareScreen	POST /api/compare	CompareService
UC-10 Filter Search Results	FR-18, FR-20, FR-22	ResultsMapListScreen	POST /api/search/{id}/view	FlatSearchService
UC-11 Authenticate User	FR-24 to FR-26	LoginScreen	POST /api/auth/login	AuthenticationService
UC-12 Update Valuation Baseline	FR-28 to FR-30, NFR-5	(none)	(scheduled)	ValuationRefreshScheduler, ValuationService
UC-13 Register Account	FR-27	SignupScreen	POST /api/auth/register	AuthenticationService
UC-14 Manage Profile	FR-33, FR-34	ProfileScreen	GET /api/profile	ProfileService
UC-15 Sign Out	FR-35	ProfileScreen	POST /api/auth/logout	AuthenticationService

2.13 Implementation Status (Lab 4)

All use cases UC-01 to UC-15 are implemented in the Spring Boot 3.3 and React application. It uses H2 by default, and PostgreSQL with Redis under the docker profile. The README maps each use case to its screen, endpoint and classes. 68 automated tests pass (re-run on 2

November 2026). These include 28 black-box tests on CpfGrantService (BB-01 to BB-28), basis path tests on AuthenticationService.login (V(G)=5) and ValuationService.ingest (V(G)=7), and end-to-end API flows.

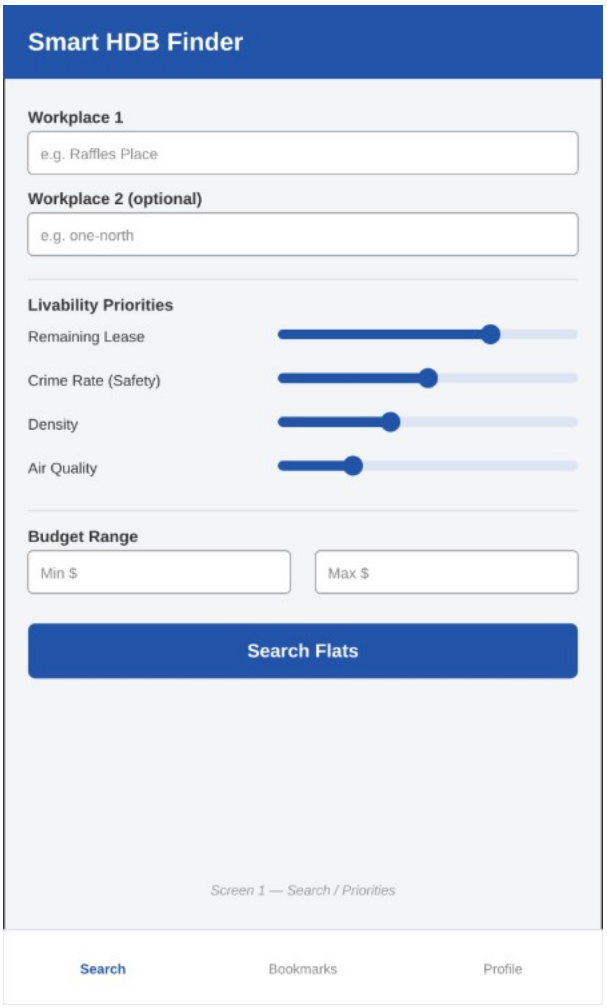
Resolved: the sliders allow tenure 5 to 25 years, down payment 25% to 100% and interest 0% to 10%. Tenure caps at 25 years (the HDB loan max) and down payment starts at 25% (the HDB minimum); the API's wider range supports bank loans later.

Data sources: the application runs in offline mode by default. Flats and resale prices come from a real data.gov.sg snapshot (1 November 2026) loaded at start-up. Travel times are estimated from distance by TravelTimeModel, PSI readings are simulated, and LTA DataMall is used only for train service alerts. Setting EXTERNAL_MODE to live switches to the OneMap, LTA DataMall and data.gov.sg APIs (FR-3, FR-28), but live mode has not yet been tested end to end.

3. External Interface Requirements

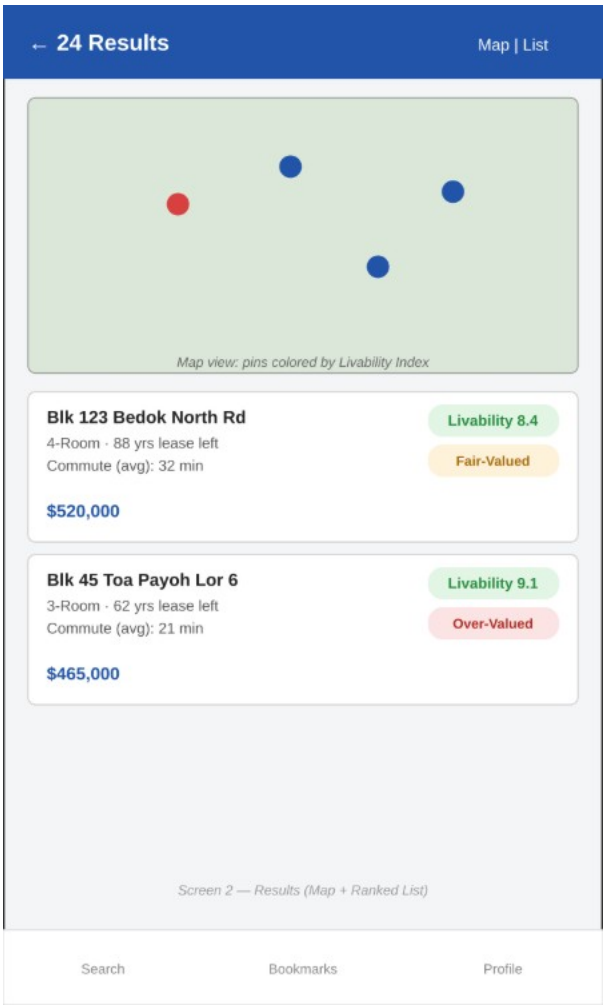
3.1 User Interfaces

Search & Livability Priorities



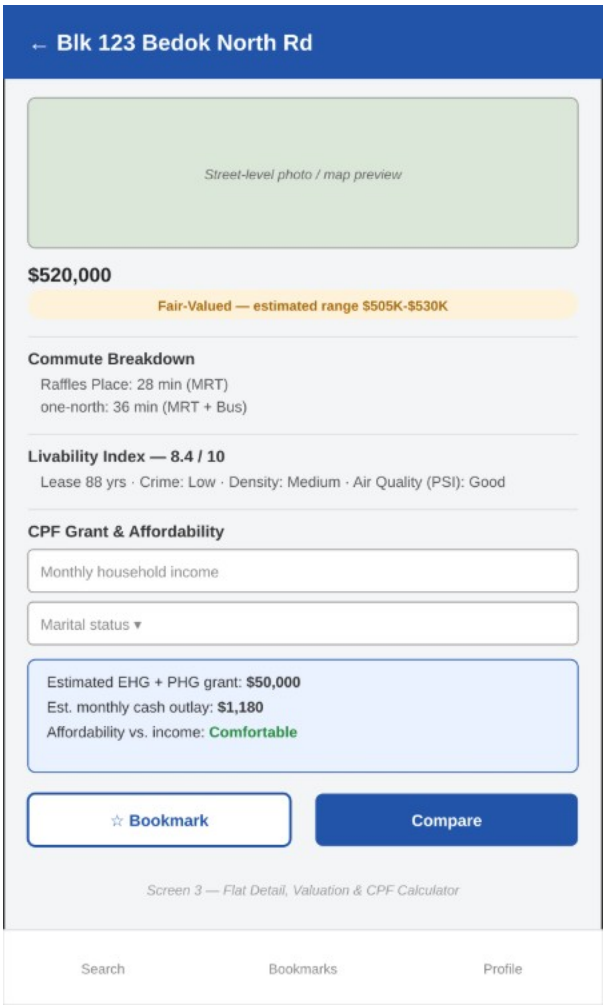
Search screen: dual workplace inputs and weighted priority sliders

Results (Map + Ranked List)



Results screen: map view with livability-coloured pins and ranked flat cards showing price and valuation label

Flat Detail, Valuation & CPF Calculator



Flat detail screen: commute breakdown, livability factors, valuation label, and CPF grant/affordability calculator

Other screens and common UI standards

The Login, Sign-Up, Bookmarks, Compare and Profile screens, and the Filter, Adjust Weights and Financial Profile panels, follow the same visual style as the three screens above. Their navigation is defined in the refined dialog map in Section 2.9.6. All screens share a bottom navigation bar with three tabs: Search, Bookmarks and Profile. Validation errors are shown next to the field they relate to. When cached data is used because an external API failed, a non-blocking warning banner is shown at the top of the screen (NFR-3). The Search screen will also show transport mode options (FR-2), which are not drawn in the low-fidelity wireframe above.

3.2 Hardware Interfaces

The software is a web-based application and does not require direct hardware interfaces. The system runs on standard web servers and is accessed through clients with modern web browsers. The following hardware considerations apply:

- Server Hardware: Standard web application server with sufficient processing power and memory to handle concurrent user loads (50 concurrent active users)

- Client Hardware: Any device capable of running supported web browsers with screen resolution ≥375px
- Network: Internet connectivity required for API calls. Typical broadband or mobile data speeds are sufficient.

3.3 Software Interfaces

The system connects to the following software components:

Component	Version	Purpose	Data In / Out
OneMap Search API	Current public API	Converts workplace addresses into map coordinates (geocoding).	Out: address text. In: latitude, longitude, postal code.
OneMap Routing API	Current public API	Returns public transport and driving routes and travel times.	Out: origin and destination coordinates, mode. In: route and duration in minutes.
OneMap Map Tiles	Current public API	Basemap shown in the MapLibre map view.	In: map tile images.
LTA DataMall	Current public API	Supplementary transport data such as train service alerts.	In: service alert records.
data.gov.sg Resale Flat Prices	Datastore API	Historical resale transactions for flats and the valuation baseline.	In: town, flat type, block, street, floor area, lease, resale price, month.
data.gov.sg NEA PSI	Real-time API	Regional PSI readings for the air quality factor.	In: region, PSI value, timestamp.
PostgreSQL with PostGIS	Current stable	Stores accounts, bookmarks, profiles, transactions, the valuation baseline and batch logs. PostGIS supports location queries.	Read and write through Spring Data JPA and Hibernate Spatial.
Redis	Current stable	Caches commute times, PSI readings and other external API responses (NFR-1, NFR-3).	Read and write through Spring Cache.
Spring Boot	Java 17 or later	Backend framework for the REST API, security, scheduling and persistence.	Not applicable (internal framework).
React	With TypeScript and Vite	Frontend framework for all screens.	Calls the backend REST API with JSON.
MapLibre GL JS	4.x	Map view with pins coloured by Livability Index.	Uses flat coordinates and OneMap tiles.

Data shared across components is exchanged as JSON over the backend REST API. The frontend never calls OneMap, LTA DataMall or data.gov.sg for data directly. Only map tiles are loaded by the browser. This keeps API keys on the server and lets the backend apply caching and the 2.0 second timeout (NFR-3).

3.4 Communications Interfaces

The system uses the following communication standards:

1. Browser to backend: HTTPS using TLS 1.3. The backend exposes a REST API that sends and receives JSON.
2. Backend to external APIs: HTTPS with a 2.0 second timeout on every call. If a call fails or times out, the cached value is used (NFR-3).
3. Authentication: after sign-in, the backend issues a signed JSON Web Token (JWT) with a short expiry. The frontend sends it in the Authorization header with each request to a protected endpoint. Sessions expire after a period of inactivity (TBD-5).
4. Cross-origin requests are only accepted from the application's own frontend address.
5. The login and registration endpoints are rate-limited to slow down password guessing.
6. Sensitive fields (household income and parental address) are never written to logs or sent to external APIs (NFR-6).
7. No e-mail or SMS is sent by this release. Password reset by e-mail is tracked as TBD-7.

4. System Features

4.1 Search Flats by Dual-Anchor Commute

4.1.1 Description and Priority

This feature allows users to search for HDB resale flats by entering one or two workplace addresses as commute anchors. The system retrieves travel-time estimates from each candidate flat to each workplace using the OneMap and LTA DataMall APIs, computes a commute score, and ranks the results by an overall match score that combines the commute score and the Livability Index. Users can then narrow the results with filters. Priority: High

4.1.2 Stimulus/Response Sequences

UC-1: Search Flats by Dual-Anchor Commute

1. User Action: User enters Workplace 1 (required) and Workplace 2 (optional)
2. System: Accepts input and validates addresses
3. User Action: User selects transport modes and search filters (budget, flat type, town/area)
4. User Action: User clicks "Search Flats"
5. System: Validates search inputs (Workplace 1 must be provided)
6. System: Retrieves candidate flats matching specified filters
7. System: Calculates commute time for each flat using Data APIs (UC-2)

8. System: Computes commute score and Livability Index, then ranks flats by overall match score
9. System: Displays ranked results in list and map views

UC-2: Calculate Commute Time (sub-function, initiated by system)

1. System: Checks the commute cache for a recent travel time
2. System: If none, sends travel-time requests to Data APIs for each candidate flat
3. Data APIs: Return travel-time estimates
4. System: Stores the result in the commute cache and provides it to the calling use case

UC-10: Filter Search Results (extends UC-1)

1. User Action: User opens the filter panel on the Results screen
2. User Action: User selects price range, flat types and/or towns
3. System: Applies the filters to the stored result set without calling the Data APIs
4. System: Displays the filtered list in the same ranked order, or a "No matching flats" message

4.1.3 Functional Requirements

ID	Requirement	Actor
FR-1	The system shall allow a user to enter Workplace 1 (required) and Workplace 2 (optional) as commute anchors for a flat search.	Home Buyer
FR-2	The system shall allow a user to select one or more preferred modes of transport for commute calculations.	Home Buyer
FR-3	The system shall retrieve travel-time estimates between each candidate flat and each workplace anchor via the OneMap and LTA DataMall APIs.	System
FR-4	The system shall compute a daily commute score for each flat for each of the two anchor travel times.	System
FR-17	The system shall display search results as both a ranked list and a map view with pins coloured by Livability Index.	System
FR-18	The system shall allow the user to filter search results by flat type (2-room to executive) and by town/planning area.	Home Buyer
FR-19	The system shall validate that Workplace 1 is provided before executing a search, and shall prompt the user if it is missing.	System
FR-20	If a search returns zero matching flats, the system shall display a "No matching flats" message and suggest which filters to relax.	System

ID	Requirement	Actor
FR-22	The system shall allow a user to specify a minimum and maximum property purchase price and filter search results accordingly.	System
FR-23	The system shall display for each search result the flat's location, flat type, transaction price, Livability Index and valuation label.	System
FR-31	The system shall rank search results by an overall match score that combines the flat's commute score and its Livability Index (formula TBD-2).	System

4.2 Adjust livability weights

4.2.1 Description and Priority

This feature allows users to customise the relative importance of four livability factors (remaining lease years, crime rate, residential density, and air quality) using sliders. The system automatically adjusts weights to maintain a total of 100% and recalculates the Livability Index, re-ranking the flat results accordingly. Priority: High

4.2.2 Stimulus/Response Sequences

UC-3: Adjust Livability Weights

1. User Action: User adjusts one or more livability weight sliders
2. System: Adjusts remaining livability weights to maintain 100% total
3. System: Recalculates Livability Index for each flat using updated weights
4. System: Re-ranks and displays flat results
5. System: If the user is signed in, saves the weights as the last-used weight profile

4.2.3 Functional Requirements

ID	Requirement	Actor
FR-5	The system shall allow a user to set relative weights (0-100%) for four livability factors: remaining lease years, crime rate, residential density, and regional air quality (PSI).	Home Buyer
FR-6	The system shall automatically adjust the remaining livability weights when a user changes one factor so that the total weight remains at 100%. The other weights share what is left in proportion to their current values. If they are all 0, they share it equally. Weights are rounded to one decimal place and any rounding difference goes to the largest of the other weights.	System

ID	Requirement	Actor
FR-7	The system shall compute a Livability Index (0-100) for each flat as a weighted sum of its normalised remaining lease, (inverted) crime rate, (inverted) density, and (inverted) regional PSI, using the user-defined weights.	System
FR-8	The system shall re-rank the flat result list whenever the user changes a livability weight.	System

4.3 View flat details, valuation and CPF calculator

4.3.1 Description and Priority

This feature allows users to view detailed information about a selected flat, including its Livability Index breakdown, fair-value pricing label and estimated range, and CPF grant eligibility with estimated monthly cash outlay. Priority: High

4.3.2 Stimulus/Response Sequences

UC-4: View Flat Details

1. User Action: User selects a flat from search results or bookmarks
2. System: Opens flat detail view and displays all available information

UC-5: View Fair-Value Pricing (extends UC-4)

1. User Action: User requests to view fair-value pricing information
2. System: Retrieves the stored rolling 6-month estate average for the flat's town and flat type (valuation baseline from UC-12)
3. System: Compares the flat's price with the average and assigns a valuation label
4. System: Displays the label, the estimated fair-value range and the baseline date

UC-6: Calculate CPF Grant and Affordability (extends UC-4)

1. User Action: User opens CPF Grant and Affordability section
2. System: Prompts for financial information (UC-7)
3. System: Estimates EHG and PHG amounts
4. System: Calculates monthly cash outlay
5. System: Displays grants, cash outlay, and loan assumptions

UC-7: Input Financial Profile (included by UC-6)

1. System: Prompts for monthly household income, marital status, parental address
2. User Action: User enters required information
3. System: Validates and makes profile available. If the user is signed in and chooses to save it, stores it encrypted.

4.3.3 Functional Requirements

ID	Requirement	Actor
FR-9	The system shall compare a flat's most recent transaction price against the rolling 6-month estate average and label it Under-Valued, Fair-Valued, or Over-Valued.	System
FR-10	The system shall allow the user to input monthly household income, marital status, and parental address to check CPF grant eligibility.	Home Buyer
FR-11	The system shall estimate the CPF Housing Grant (Family Grant: $80,000 for 2- to 4-room, $50,000 for 5-room or larger, household income up to $16,000; Singles Grant: $40,000 / $25,000, income up to $8,000), the Enhanced CPF Housing Grant and the Proximity Housing Grant. Amounts and ceilings come from a configuration file (grant-rules.yml, effective 24 Aug 2026), so the code holds no fixed amounts. Every grant assumes a first-time buyer who is a Singapore citizen.	System
FR-12	The system shall display the loan assumptions used to estimate the monthly cash outlay.	System
FR-13	The system shall allow the user to adjust selected affordability assumptions, including loan tenure and down payment, before recalculating the estimated monthly cash outlay.	Home Buyer
FR-32	The system shall display an estimated fair-value price range for the selected flat, together with the estate average and the date of the valuation baseline used.	System

4.4 Manage bookmarks and compare flats

4.4.1 Description and Priority

This feature allows users to bookmark flats for later retrieval and compare two or more bookmarked flats side-by-side on key metrics. Priority: Medium

4.4.2 Stimulus/Response Sequences

UC-8: Bookmark Flat

1. User Action: User selects "Bookmark" for a flat
2. System: If the user is not signed in, invokes UC-11 first
3. System: Saves flat to user's bookmarks

Alternative: Flat already bookmarked

1. User Action: User chooses to remove from bookmarks
2. System: Removes flat from bookmarks

UC-9: Compare Bookmarked Flats

1. User Action: User opens bookmarked flats list
2. User Action: User selects two or more flats
3. User Action: User selects "Compare"
4. System: Displays selected flats side-by-side with differences highlighted

4.4.3 Functional Requirements

ID	Requirement	Actor
FR-14	The system shall allow a user to bookmark a flat for later retrieval.	Home Buyer
FR-15	The system shall allow a user to remove a previously bookmarked flat from their saved bookmarks.	Home Buyer
FR-16	The system shall allow a user to compare two or more bookmarked flats side-by-side on price, commute, livability, and valuation.	Home Buyer
FR-21	The system shall persist a signed-in user’s bookmarks and last-used weight profile across sessions.	System

4.5 Authenticate User

4.5.1 Description and Priority

This feature verifies a Home Buyer’s identity before granting access to features that require a persistent, personal record, such as bookmarking flats, comparing bookmarks, or viewing/updating a saved profile. It is invoked automatically whenever an unauthenticated user attempts one of these actions. Users without an account can register (UC-13). Priority: High

4.5.2 Stimulus/Response Sequences

UC-11: Authenticate User

1. User Action: User attempts an action requiring an identified session (e.g., selects "Bookmark" on a flat)
2. System: Detects that no active authenticated session exists
3. System: Displays login screen, prompting for credentials
4. User Action: User enters credentials and submits
5. System: Validates credentials against the stored salted password hash
6. System: Establishes an authenticated session with a secure, time-limited session token
7. System: Returns control to the calling use case (e.g., completes the bookmark action)

UC-13: Register Account (extends UC-11)

1. User Action: User selects "Sign Up" on the login screen
2. System: Collects email, password and consent to the personal data notice
3. System: Validates the details and creates a new account
4. System: Proceeds to Step 6 of UC-11 above

4.5.3 Functional Requirements

ID	Requirement	Actor
FR-24	The system shall require successful authentication before allowing access to bookmarking, comparison, or profile features.	System
FR-25	The system shall validate submitted credentials and, on success, establish a time-limited authenticated session.	System
FR-26	The system shall lock an account after a configurable number of consecutive failed login attempts and inform the user.	System
FR-27	The system shall allow a user with no existing account to register a new account before authenticating.	Home Buyer

4.6 Update Valuation Baseline (Batch Refresh)

4.6.1 Description and Priority

This feature automatically refreshes the fair-value pricing baseline used by the Valuation feature (Section 4.3) by ingesting the latest historical resale transaction data from data.gov.sg on a recurring 24-hour cycle, without requiring any Home Buyer interaction. Priority: Medium

4.6.2 Stimulus/Response Sequences

UC-12: Update Valuation Baseline (24h Batch Refresh)

1. System (Scheduler): Triggers the batch refresh job at the scheduled 24-hour interval
2. System: Queries data.gov.sg for new or updated resale transaction records
3. System: Recomputes the rolling 6-month estate average price for each affected town/flat type
4. System: Stores the updated valuation baseline and records the refresh timestamp

Alternative: External API failure

1. System: Detects that data.gov.sg is unavailable or returns an error
2. System: Retains the previous valuation baseline
3. System: Logs the failed refresh attempt and raises an internal alert

4.6.3 Functional Requirements

ID	Requirement	Actor
FR-28	The system shall automatically query data.gov.sg every 24 hours to ingest new resale transaction records.	System
FR-29	The system shall recompute the rolling 6-month estate average price used for valuation labelling after each successful batch refresh.	System
FR-30	If a scheduled batch refresh fails, the system shall retain the previous valuation baseline and log the failure for follow-up.	System

4.7 Manage Profile and Sign Out

4.7.1 Description and Priority

This feature allows a signed-in Home Buyer to view their account and saved preferences, update or delete their saved financial profile, and sign out. Priority: Medium

4.7.2 Stimulus/Response Sequences

UC-14: Manage Profile

1. User Action: User selects the Profile tab (UC-11 is invoked if not signed in)
2. System: Displays account email, saved weight profile and saved financial profile summary
3. User Action: User edits or deletes the saved financial profile
4. System: Validates and saves the change, or deletes the stored data

UC-15: Sign Out

1. User Action: User selects "Sign Out" on the Profile screen
2. System: Invalidates the session and clears personal data from the browser
3. System: Returns the user to the Search screen as a guest

4.7.3 Functional Requirements

ID	Requirement	Actor
FR-33	The system shall allow a signed-in user to view their account details, saved weight profile and saved financial profile.	Home Buyer
FR-34	The system shall allow a signed-in user to update or permanently delete their saved financial profile.	Home Buyer
FR-35	The system shall allow a signed-in user to sign out, which shall end the session on the server and return the user to the Search screen as a guest.	Home Buyer

5. Other Nonfunctional Requirements

5.1 Performance Requirements

ID	Requirement
NFR-1	The system shall return filtered and ranked results for up to 500 flats within ≤ 3 seconds at the 95th percentile, utilising cached travel times, in-memory result sets and indexed database queries.
NFR-3	If an external API call fails or exceeds a 2.0-second timeout, the system shall fall back to cached data within 500 ms and display a non-blocking warning informing the user that data may be outdated.
NFR-7	The backend shall sustain a sustained load of 50 concurrent active users executing search and weight-adjustment queries with an error rate <1% and average latency degradation ≤15%.

Rationale: These performance requirements ensure a responsive user experience even during peak usage and maintain system reliability when external services are unavailable.

5.2 Safety Requirements

The system does not directly control physical equipment or processes and therefore has minimal safety requirements. However, the following safety considerations apply:

- The system shall not take actions that could cause financial harm due to incorrect calculations
- All financial calculations (grants, monthly outlay) shall be clearly marked as estimates
- Users shall be clearly informed that all data is for reference and final decisions should be made after consulting official sources
- The system shall include clear disclaimers that grant eligibility and amounts are estimates and subject to final approval by HDB

5.3 Security Requirements

ID	Requirement
NFR-6	The system shall encrypt all sensitive user data (including household income and parental addresses) both in transit (via HTTPS) and at rest in the database, and shall ensure this data is neither exposed in server logs nor shared with external third-party APIs.

Additional security requirements:

- User authentication shall be required to access bookmarks and saved profiles
- Passwords shall be stored only as salted hashes (bcrypt), never in plain text
- Session management with appropriate timeout and secure token handling
- Input validation and sanitisation to prevent injection attacks
- Role-based access control (Home Buyer vs. administrator). Administrator features are not in scope for this release (TBD-6).
- Regular security audits and penetration testing

5.4 Software Quality Attributes

Attribute	Requirement
Availability	The system shall maintain ≥99.0% service availability during active operational hours, excluding pre-notified scheduled maintenance windows (NFR-4).
Data Freshness	An automated batch job shall query data.gov.sg every 24 hours to ingest new resale transaction records and update the rolling 6-month valuation baseline (NFR-5).
Portability	The web application UI shall render correctly and maintain touch/input usability on the latest two major releases of Google Chrome, Apple Safari, and Microsoft Edge across desktop (≥1280px) and mobile (≥375px) viewports (NFR-8).
Usability	At least 85% of novice test participants shall successfully configure search inputs, execute a search, and view ranked results within 3 minutes without operator intervention during testing (NFR-2).
Reliability	The system shall handle external API failures gracefully with fallback to cached data and non-blocking warnings (NFR-3).
Maintainability	The system shall be designed with a modular, layered architecture so that each layer can be updated with little impact on the others.
Testability	All functional requirements (FR-1 through FR-35) shall be independently verifiable and traceable to test cases.

5.5 Business Rules

The following business rules govern the operation of the system:

1. Weight Allocation: The sum of all livability factor weights must equal 100%. When a user adjusts one weight, the remaining weights are automatically adjusted to maintain this balance.

2. Search Validation: Workplace 1 is required for all searches. If not provided, the system shall prompt the user.
3. Valuation Classification: A flat is classified as Under-Valued when its price is significantly below the rolling 6-month estate average, Fair-Valued when its price is near the average, and Over-Valued when its price is significantly above the average. The exact band is TBD-1.
4. Grant Estimation: CPF Housing Grant, EHG and PHG eligibility and amounts are estimated based on monthly household income (for the CPF Housing Grant and EHG), flat type (for the CPF Housing Grant), parental address proximity (for PHG) and marital status.
5. Data Freshness: Valuation baselines are updated every 24 hours through an automated batch job.
6. Comparison Requirement: At least two bookmarked flats must be selected for side-by-side comparison.
7. Ranking: Search results are ranked by an overall match score that combines commute score and Livability Index (TBD-2).

6. Other Requirements

Database requirements. All persistent data is stored in a single PostgreSQL database with the PostGIS extension. Schema changes are managed with versioned migration scripts kept in the GitHub repository. Indexes are created on town, flat type and price to meet NFR-1. The database is backed up daily. Sensitive columns are encrypted at rest (NFR-6).

Legal requirements. The system must follow the PDPA. Users must accept a personal data consent notice at sign-up, and must be able to delete their saved financial profile at any time (FR-34). Data from data.gov.sg is used under the Singapore Open Data Licence, and the OneMap logo and attribution must be shown on the map view as required by its terms of use.

Internationalisation requirements. The interface is in English only. All prices are in Singapore dollars and all times are in Singapore time (SGT).

Reuse objectives. The external API clients sit behind a single gateway interface, so a data provider can be replaced without changing the rest of the system.

Appendix A: Data Dictionary

Term	Description	Key Attributes	Relationships
User (Home Buyer)	A person searching for an HDB resale flat.	userId, income, maritalStatus, parentAddress	User 1 ── 0..1 FinancialProfile User 1 ── 0..* SearchCriteria User 1 ── 0..* Bookmark User 1 ── 0..1 WeightProfile User 1 ── 0..* Session
HDB	Stands for Housing and Development Board and is the organisation for Singapore's public housing	-	-
Flat	An HDB flat (block and flat type) that appears in recent resale transactions. It is not a live listing.	flatId, block, street, town, flatType, leaseRemaining, floorArea, latitude, longitude, region	Flat 1 ── 1..* Transaction Flat 1 ── 0..* LivabilityIndex Flat 1 ── 1 ValuationLabel Flat 1 ── 0..* Bookmark Flat 1 ── 0..* CommuteRoute Flat 1 ── 0..* CommuteScore Flat 0..* ── 0..* School
Workplace	A commute anchor address entered by the user.	address, latitude, longitude	Workplace 1..2 ── 1 SearchCriteria Workplace 1 ── 0..* CommuteRoute
CommuteRoute	A computed travel path/time between a Flat and a Workplace.	originId, destinationId, mode, durationMinutes	CommuteRoute 0..* ── 1 Flat CommuteRoute 0..* ── 1 Workplace CommuteRoute 1..* ── 1 CommuteScore
LivabilityIndex	A weighted 0-100 score describing a Flat's livability, based on dwelling/environmental quality factors, excluding amenities. It is calculated each time from the flat's factor scores and the current user's weights, so it is not stored and the same flat can score differently for different users.	leaseScore, crimeScore, densityScore, airQualityScore, weightedTotal	LivabilityIndex 0..* ── 1 Flat LivabilityIndex 0..* ── 1 WeightProfile LivabilityIndex 1..* ── 1 CrimeRecord LivabilityIndex 1..* ── 1 DensityRecord LivabilityIndex 1..* ── 1 AirQualityReading

Term	Description	Key Attributes	Relationships
WeightProfile	User-defined relative importance of each livability factor.	leaseWeight, safetyWeight, densityWeight, airQualityWeight	WeightProfile 0..1 ── 1 User WeightProfile 1 ── 0..* LivabilityIndex
Transaction	A historical resale transaction record for a block/estate.	transactionId, flatId, resalePrice, transactionDate	Transaction 1..* ── 1 Flat Transaction 1..* ── 1 ValuationLabel
ValuationLabel	Classification of a Flat's price vs. estate average.	label (Under/Fair/Over), estateAveragePrice	ValuationLabel 1 ── 1 Flat ValuationLabel 1 ── 1..* Transaction
FinancialProfile	User-entered financial details used for grant calculation.	monthlyIncome, maritalStatus, parentProximityKm	FinancialProfile 0..1 ── 1 User FinancialProfile 1 ── 1 CPFGrant
CPFGrant	Estimated CPF (Central Provident Fund) housing grant amount and resulting cash outlay.	chgAmount, ehgAmount, phgAmount, monthlyOutlay	CPFGrant 1 ── 1 FinancialProfile CPFGrant 0..* ── 1 Flat
School	A MOE primary school location, shown as optional convenience info (not part of the core Livability Index).	schoolId, name, latitude, longitude, schoolAddress, schoolType	School 0..* ── 0..* Flat (amenity reference only)
CrimeRecord	Aggregated crime statistics for a planning area, sourced from SPF Annual Crime Brief.	planningArea, crimeRate, year	CrimeRecord 1 ── 1..* LivabilityIndex
AirQualityReading	Regional PSI (Pollutant Standards Index) reading from NEA (National Environment Agency), used as an air-quality proxy.	region, psiValue, timestamp	AirQualityReading 1 ── 1..* LivabilityIndex
DensityRecord	Residential density for a subzone/block, sourced from URA/SingStat.	subzone, unitsPerSubzone	DensityRecord 1 ── 1..* LivabilityIndex

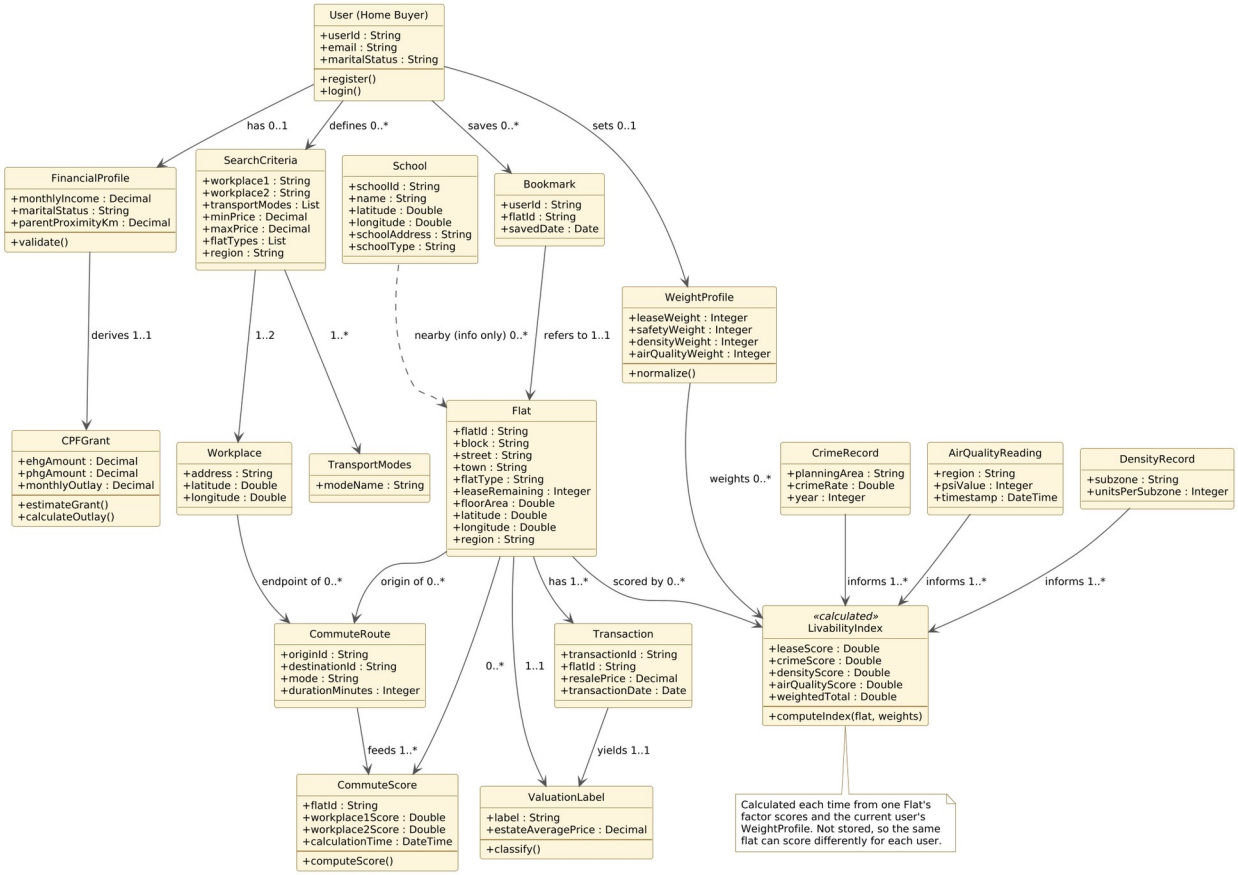
Term	Description	Key Attributes	Relationships
Bookmark	A saved reference from a User to a Flat.	userId, flatId, savedDate	Bookmark 0..* ── 1 User Bookmark 0..* ── 1 Flat
SearchCriteria	User-selected filters and commute preferences used to search for flats.	workplace1, workplace2, transportModes, minPrice, maxPrice, flatTypes, region	SearchCriteria 0..* ── 1 User SearchCriteria 1 ── 1..2 Workplace SearchCriteria 0..* ── 1..* TransportModes SearchCriteria 0..* ── 0..* Flat SearchCriteria 1 ── 0..* CommuteRoute
TransportModes	A transport method selected by the user for commute calculations.	modeName	TransportModes 1..* ── 0..* SearchCriteria TransportModes 1 ── 0..* CommuteRoute
CommuteScore	A score representing the combined commute effort required to go from a flat to the user's workplace.	flatId, workplace1, workplace2, calculationTime	CommuteScore 0..* ── 1 Flat CommuteScore 1 ── 1..* CommuteRoute CommuteScore 0..* ── 1..2 Workplace CommuteScore 0..* ── 1 SearchCriteria
Session	An authenticated user's active login session, used to control access to bookmarking and profile features.	sessionId, userId, createdAt, expiresAt, token	Session 0..* ── 1 User
ValuationBatchJob	A scheduled job that runs every 24 hours to retrieve new resale transaction data from data.gov.sg and refresh the rolling 6-month valuation baseline.	jobId, runTimestamp, status, recordsIngested	ValuationBatchJob 1 ── 0..* Transaction ValuationBatchJob 1 ── 0..* ValuationLabel

Appendix B: Analysis Models

B.1 Class Diagram
B.2 Key Boundary Classes and Control Classes
B.3 Initial Dialog Map
B.4 Use Case Diagram

Class Diagram of Entity Classes

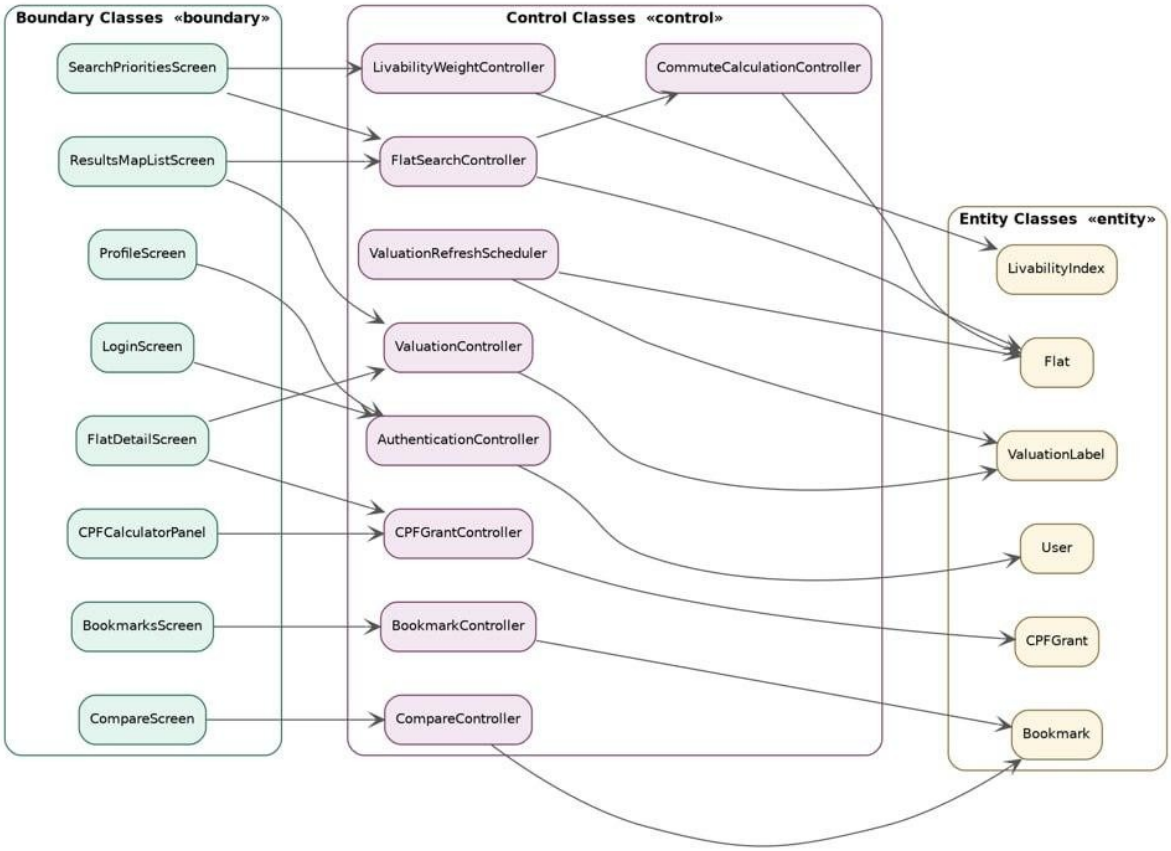
The class diagram below models the entity classes from Appendix A (Data Dictionary) of the SRS, together with their key attributes, operations, and associations. It reflects how a User's search criteria, workplace anchors and weight profile drive Flat-level commute scores, valuation labels and Livability Index scores, and how bookmarks and financial profiles support UC-6 through UC-9.



Class Diagram B.1

Key Boundary Classes and Control Classes

Applying the Boundary-Control-Entity (BCE) pattern to the use cases and the UI mockups in the SRS, the following boundary and control classes are identified. Boundary classes correspond to the three numbered screens (and supporting screens) shown in the SRS wireframes. Control classes mediate between those screens and the entity classes.



Key Boundary Classes and Control Classes B.2

B.2.1 Boundary Class

Class	Stereotype	Responsibility
LoginScreen	Boundary	Collects credentials and initiates authentication. Entry point before bookmarking or profile features are accessible.
SearchPrioritiesScreen	Boundary	Screen 1: collects Workplace 1/2, transport modes, budget range and livability priority sliders.
ResultsMapListScreen	Boundary	Screen 2: presents ranked flats as a livability-coloured map and a scrollable list of flat cards.

Class	Stereotype	Responsibility
FlatDetailScreen	Boundary	Screen 3: shows a flat's photo/map preview, price, valuation label, commute breakdown, and livability factors.
CPFCalculatorPanel	Boundary	Embedded panel on FlatDetailScreen that collects income/marital status and displays grant and cash-outlay results.
BookmarksScreen	Boundary	Lists saved flats. Lets the user select flats to compare or remove.
CompareScreen	Boundary	Displays two or more bookmarked flats side-by-side on price, commute, livability and valuation.
ProfileScreen	Boundary	Displays account details and provides sign-out.

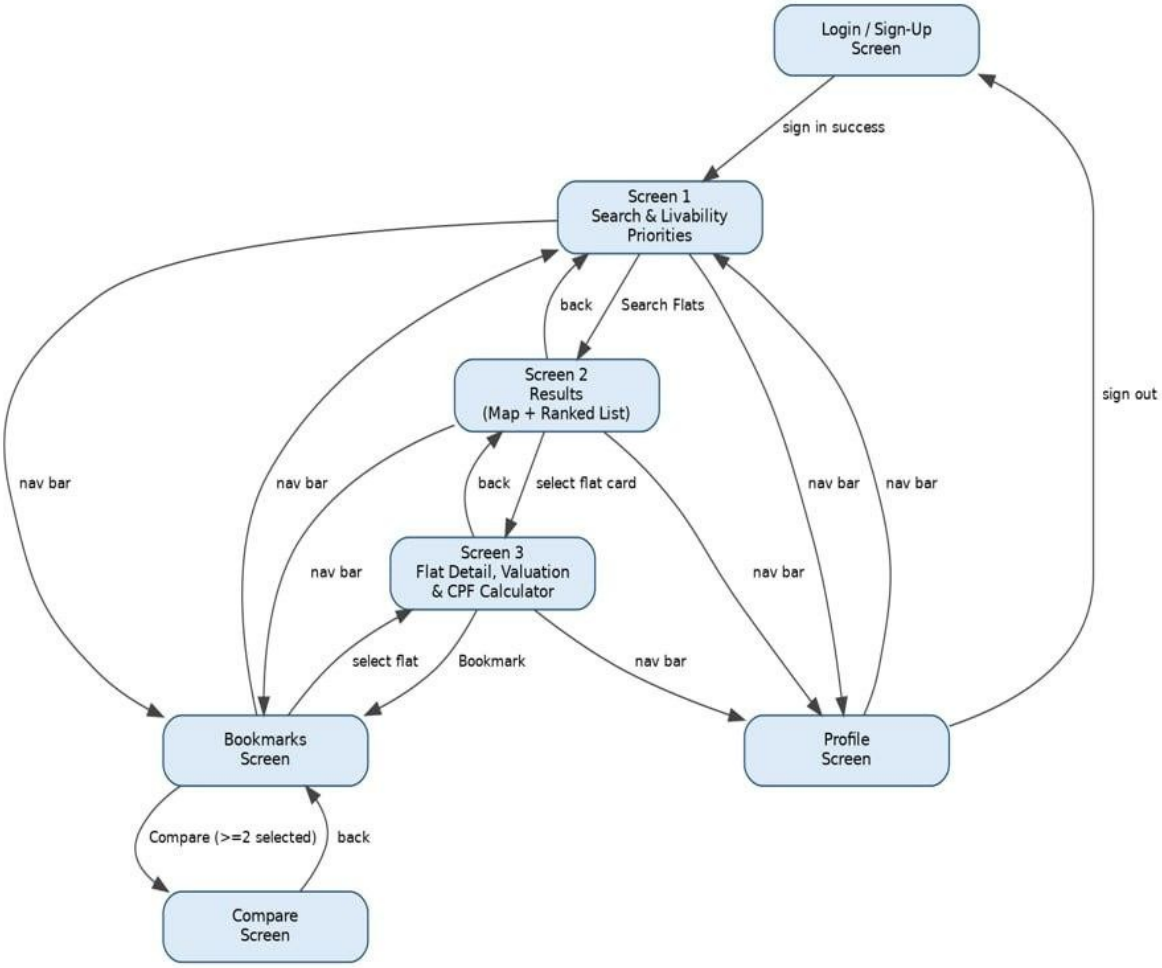
B.2.2 Control Class

Class	Stereotype	Responsibility
AuthenticationController	Control	Validates credentials, manages sessions/tokens and timeouts, and authorises access to bookmarking and profile features.
FlatSearchController	Control	Coordinates UC-1: validates Workplace 1, applies filters, and orchestrates CommuteCalculationController and ranking.
CommuteCalculationController	Control	Coordinates UC-2: calls OneMap/LTA DataMall APIs, applies the 2.0 s timeout/cache fallback, and returns travel times.
LivabilityWeightController	Control	Coordinates UC-3: rebalances slider weights to total 100%, recomputes LivabilityIndex, and triggers re-ranking.
ValuationController	Control	Coordinates UC-5: retrieves the rolling 6-month estate average and assigns the Under/Fair/Over-Valued label.

Class	Stereotype	Responsibility
CPFGrantController	Control	Coordinates UC-6/UC-7: validates the financial profile and computes EHG/PHG estimates and monthly cash outlay.
BookmarkController	Control	Coordinates UC-8: adds/removes bookmarks and persists them against the signed-in user.
CompareController	Control	Coordinates UC-9: assembles the side-by-side comparison of two or more bookmarked flats.
ValuationRefreshScheduler	Control	Runs the 24-hour automated batch job that ingests new data.gov.sg transactions and refreshes the valuation baseline (NFR-5).

Initial Dialog Map

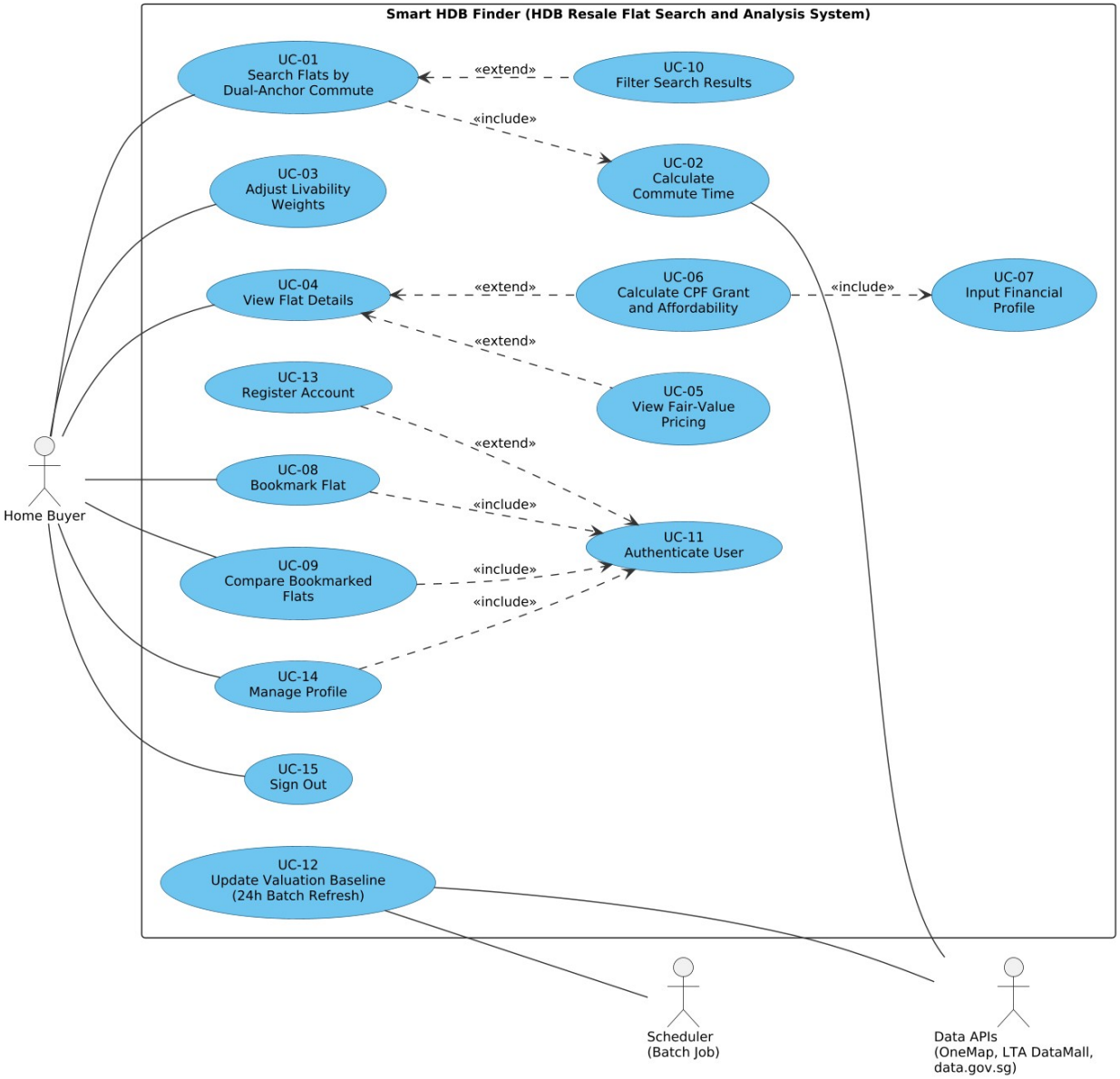
The dialog map traces navigation between the boundary classes (screens), based on the bottom navigation bar (Search / Bookmarks / Profile) shown in the SRS wireframes and the actions available on each screen. It is refined in Section 2.9.6.



Initial Dialog Map B.3

Use Case Diagram

The diagram below is the complete use case model, updated in version 1.2. It fixes the direction of the extend relationships (UC-05 and UC-06 now extend View Flat Details), changes Filter Search Results to an extension of UC-01, makes Compare Bookmarked Flats a standalone use case, and adds Register Account (UC-13), Manage Profile (UC-14) and Sign Out (UC-15). Full descriptions are in the Use Case document, version 1.2.



Use case Diagram B.4

Appendix C: To Be Determined List

These items were open in earlier versions. The Status column shows how each one was settled by version 1.4. Only TBD-9 is still open.

ID	Item	Where it is used	Resolution	Status
TBD-1	Price band for Under-, Fair- and Over-Valued labels	FR-9, Business Rule 3	Fair-Valued within 5% of the rolling average. Set in application.yml (fair-band-percent) and applied by ThresholdValuationStrategy.	Resolved
TBD-2	Overall match score formula	FR-31, Business Rule 7	Equal weight on commute score and Livability Index, both scaled to 0-100 (FlatSearchService).	Resolved
TBD-3	Default livability weights	UC-03, FR-5	25% for each of the four factors (WeightProfile default).	Resolved
TBD-4	Failed login limit and lock time	FR-26, UC-11	5 failed attempts, then a 15 minute lock (application.yml).	Resolved
TBD-5	Session (JWT) expiry	FR-25, Section 3.4	Access token valid for 30 minutes and renewed while the user is active, with an 8 hour maximum (application.yml).	Resolved
TBD-6	Administrator features	Section 5.3	Not built in this release. The user role is kept in the design for later.	Out of scope
TBD-7	Password reset	Section 3.4	Not built in this release, because it needs an e-mail service.	Out of scope
TBD-8	Hubs used for the pre-computed commute matrix	NFR-1	No pre-computed matrix was built. Travel times are cached and each search is kept in SearchSessionStore, which meets NFR-1 (Section 2.8.3).	Replaced
TBD-9	Managed platform for the Docker deployment	Section 2.4	Not chosen yet. The application runs locally with docker compose (Section 2.8.2).	Open

Source: http://www.frontiernet.net/~kwiegers/process_assets/srs_template.doc